A ZOO Deep Learning Survey

Francois Chaubard
Stanford University
`fchaubar@stanford.edu`

Abstract

Zero-order optimization (ZOO) trains a network from loss evaluations alone, without back-propagating errors: the natural training rule for neuromorphic and analog hardware, non-differentiable systems, biologically plausible learning and memory-constrained fine-tuning. Many ZOO rules exist but are rarely compared under one budget on the same networks. We benchmark nine backprop-free training rules—SPSA and its curvature-preconditioned variant 1.5-SPSA, random search, classic, activity-based, iterative and decorrelated node perturbation, curvature-preconditioned activity perturbation (1.5-ANP), and the forward-forward algorithm—against backpropagation on four ≈40k-parameter architectures (an MLP on MNIST, a ConvNet on CIFAR-10, a character-level LSTM and Transformer on Penn Treebank), with the same forward passes per step (backpropagation, the reference, takes one forward and one backward pass per step), one GPU-hour, the same Adam update and a probe scale tied to the learning rate. A screening sweep over five scales selects each method's scale per task; the selected configurations are replicated with three seeds (two for two pairs), evaluated every minute and reported at their best validation checkpoint and at the end of the hour, with a backpropagation reference at the same granularity, a grid that decouples the probe scale from the step size, the two curvature-preconditioned rules under their own recipes, and an estimator-alignment measurement. The screening's first lesson is that the step size, not the probe scale, decides: at $\varepsilon = 10^{-4}$ eight of the ten rules improve on both image tasks and the weight-space rules close about half of their gap. Iterative node perturbation (INP) is the most consistent zero-order rule: best mean rank, best probe method on the LSTM and within 0.03 bits of the best on the Transformer (2.91 and 3.14 bits per character against 1.80 and 1.84 for backpropagation), and within seed noise of the best on the image tasks together with classic and activity-based node perturbation (96.9 % on MNIST and 63–65 % on CIFAR-10, against 97.0 and 69.2 % for backpropagation). Curvature preconditioning leaves SPSA unchanged under Adam and does not help it under SGD; 1.5-ANP is usable on the sequence models where ANP is not, but its $\alpha = 0$ control shows that the curvature weighting is not the ingredient responsible. At ten and a hundred times the parameter count (one seed) the node rules hold up and classic node perturbation becomes the best probe rule, while the weight-space rules and the curvature-weighted rules fall behind. Code and run logs: `https://github.com/Fchaubard/zero-order-ablations`.

1 Introduction

Backpropagation [Rumelhart et al., 1986] is the default way to train neural networks, but it is not always available. Neuromorphic and analog accelerators implement a forward pass natively and may have no mechanism to transport exact error signals backwards through the substrate; systems with non-differentiable components (simulators, hardware in the loop, discrete decisions, black-box APIs) expose only a loss value; neuroscience asks which learning rules a brain could plausibly implement

39th Conference on Neural Information Processing Systems (NeurIPS 2025).

with local information; and fine-tuning of very large models is increasingly limited by the activation memory that backpropagation must store [Malladi et al., 2023]. In all of these settings the learner can evaluate the loss at perturbed parameters or activities but cannot differentiate it; *zero-order optimization* (ZOO) estimates a descent direction from such evaluations alone.

The ZOO literature is old and broad: random search [Rastrigin, 1963, Matyas, 1965, Schumer and Steiglitz, 1968], simultaneous perturbation stochastic approximation [Spall, 1992, 1998], Gaussian smoothing and its convergence theory [Nesterov and Spokoiny, 2017, Duchi et al., 2015, Flaxman et al., 2005], evolution strategies [Wierstra et al., 2014, Hansen et al., 2003, Salimans et al., 2017, Mania et al., 2018], node perturbation [Widrow and Lehr, 1990, Werfel et al., 2003, Fiete and Seung, 2006, Hiratani et al., 2022] and its recent activity-based and decorrelated variants [Dalm et al., 2026], curvature-preconditioned variants [Chaubard, 2025b,a], and local learning rules such as forward-forward [Hinton, 2022]. Each paper demonstrates its method on its own architecture, budget and optimizer; a practitioner who must train without backpropagation is left with the question this paper answers empirically: *which ZOO method wins, on standard small architectures, when every method is given the same budget?*

The protocol (Sections 3 and 4) fixes four ≈40k-parameter models on four standard tasks, gives every probe method the same forward passes per step, one hour of training on one GPU and the same Adam update, ties the probe scale to the learning rate so that a single scalar ε is swept per method, selects ε in a single-seed *screening* sweep and *replicates* the selected configurations with three seeds, reported at the best validation checkpoint (the quantity a practitioner would keep) and at the end of the hour. Ablations test the protocol itself: a backpropagation reference at the same granularity, a grid that decouples the probe scale from the step size, the curvature-preconditioned rules under their own recipes, and every estimator's cosine with the true gradient.

Contributions. (i) A fully specified two-stage protocol under which ten training rules—backpropagation, SPSA, 1.5-SPSA, random search, four node-perturbation variants, 1.5-ANP and forward-forward—are compared on four architectures with identical forward-pass, wall-clock and optimizer budgets, three seeds (two for two pairs, Table 15), and selection on validation data only. (ii) One implementation of every rule on a shared *perturbable layer* abstraction (Appendix G), so that every method drives the same models, including an LSTM and a pre-LN Transformer; the two curvature-preconditioned rules are implemented from their code repository and specification. (iii) The resulting comparison, backpropagation reference, probe-scale / step-size decoupling and estimator-alignment diagnostic (Tables 1–8 and Figures 4–7), with code, configurations and every run log released.

2 Related Work

Weight-space perturbation. SPSA [Spall, 1992, 1998] estimates a gradient from two loss evaluations at $\boldsymbol{\theta} \pm \sigma\mathbf{z}$; with Gaussian $\mathbf{z}$ it is the gradient of the Gaussian-smoothed loss [Nesterov and Spokoiny, 2017], with convex [Duchi et al., 2015] and bandit [Flaxman et al., 2005] complexity theory (survey: Liu et al., 2020), analog-VLSI origins [Jabri and Flower, 1992, Cauwenberghs, 1992, Flower and Jabri, 1992], evolution strategies [Wierstra et al., 2014, Hansen et al., 2003, Hansen, 2016] that compete with policy gradients [Salimans et al., 2017, Mania et al., 2018], relate to SGD [Zhang et al., 2017] and apply to supervised learning [Lenc et al., 2019], adaptive-moment updates [Chen et al., 2019] and directional-derivative learning [Silver et al., 2022]. MeZO [Malladi et al., 2023] fine-tunes billion-parameter language models with two-point SPSA in place; 1.5-SPSA [Chaubard, 2025b] adds a per-direction three-point curvature estimate to damp the step in sharp regions. DeepZero [Chen et al., 2024] and Zhang et al. [2024] benchmark estimators on large pretrained models; we vary the estimator *and* the architecture at a scale where every method runs to its one-hour limit, and our SPSA, 1.5-SPSA and RS are the two-point, curvature-preconditioned and best-of-K instances of this family.

Node perturbation. Node perturbation injects noise into the units and correlates the loss change with it, a local update [Widrow and Lehr, 1990, Werfel et al., 2003, 2005, Fiete and Seung, 2006] whose stability and weight diffusion Hiratani et al. [2022] analysed; Züge et al. [2023] found weight perturbation often similar or better on temporally extended tasks, revisited here. Dalm et al. [2026] recast it as a directional derivative (*iterative* NP), proposed *activity-based* NP from the measured activity change, and added the input decorrelation of Ahmad et al. [2023], which also accelerates backpropagation [Dalm et al., 2024, Ahmad, 2024]; 1.5-ANP [Chaubard, 2025a] adds a three-point curvature preconditioner to central-difference activity perturbation; we implement all five.

Local rules and benchmarks. Forward-forward [Hinton, 2022], our representative local rule, replaces the backward pass by two forward passes with positive and negative data and a local "goodness" objective per layer. Global-error broadcasting [Clark et al., 2021], feedback alignment [Lillicrap et al., 2016, Nøkland, 2016] and synthetic gradients [Jaderberg et al., 2017] still need a learned or random backward signal, and forward gradients [Baydin et al., 2022, Ren et al., 2023] a Jacobian-vector product, so they are outside the zero-order scope. Controlled comparisons exist for the feedback-alignment family [Bartunov et al., 2018, Launay et al., 2020, Sanfiz and Akrout, 2021]; we are not aware of one that puts weight perturbation, the node-perturbation family and forward-forward on the same models under one budget (unabridged: Appendix D).

3 Methods

3.1 Problem setup and the shared update rule

Let $\boldsymbol{\theta} \in \mathbb{R}^P$ be the parameters of a network with L perturbable layers and $\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N}\sum_{n=1}^{N} \ell_n(\boldsymbol{\theta})$ the mini-batch cross-entropy loss, where ℓ_n is the per-example loss (for language modelling, the per-sequence mean of the $T = 64$ next-character cross-entropies). Every method writes an estimate $\hat{\mathbf{g}} \approx \nabla_{\boldsymbol{\theta}}\mathcal{L}$ into the gradient slot of every parameter—backpropagation the exact gradient, the zero-order methods their estimator, forward-forward the gradient of its local objectives—and the same optimizer consumes it: Adam [Kingma and Ba, 2015] with $(\beta_1, \beta_2) = (0.9, 0.99)$, learning rate η and no schedule (Eq. (1), Appendix A). Every probe method has a second scalar, the probe scale σ: the standard deviation of the parameter perturbation for the weight-space methods and of the node noise for the node-perturbation family; all probes are drawn from $\mathcal{N}(\mathbf{0}, \mathbf{I})$ and scaled by σ. The protocol *ties* the two, $\eta = \sigma = \varepsilon$, so that one scalar is swept per method; Section 5.3 measures what the tie costs. Because Adam normalises $\hat{\mathbf{g}}$ by a running estimate of its own magnitude, Eq. (1) is approximately invariant to a constant rescaling of the estimator: the normalisation constants of the estimators affect the first few hundred steps and the relative weighting of estimates across steps, not the asymptotic step size, so the comparison is insensitive to the different conventions in the literature.

3.2 Optimizers

Appendix A has full definitions (Eqs. (2)–(9)) and the budget (Table 2); here, estimator and cost.

SPSA. With $K = 8$ antithetic pairs of Gaussian probes $\mathbf{z}_k$, $\hat{\mathbf{g}}_{\text{SPSA}}$ averages the central differences $[\mathcal{L}(\boldsymbol{\theta} + \sigma\mathbf{z}_k) - \mathcal{L}(\boldsymbol{\theta} - \sigma\mathbf{z}_k)]\,\mathbf{z}_k/2\sigma$ (Eq. (2)): the Gaussian-smoothing estimator of Nesterov and Spokoiny [2017] and multi-direction SPSA [Spall, 1992] with Gaussian rather than Rademacher directions, perturbed in place as in MeZO [Malladi et al., 2023]; 16 passes, no clean pass.

1.5-SPSA. 1.5-SPSA [Chaubard, 2025b] adds a clean pass (17 passes) and divides each central difference by $c_k = \max(\hat{\kappa}_k^{\alpha}, \lambda)$, $\hat{\kappa}_k$ the three-point second difference along $\mathbf{z}_k$ ($\alpha = 0.1$, $\lambda = 1$; Eq. (3)). Under Adam c_k is inert (Section 5.4), so Section 5.4 also runs the published recipe (Rademacher directions, plain SGD, $\eta = \sigma$); we use Gaussian directions throughout.

Random search. Random search [Rastrigin, 1963, Matyas, 1965, Schumer and Steiglitz, 1968] evaluates the clean loss and $K = 15$ candidates $\boldsymbol{\theta} + \sigma\mathbf{z}_k$ and keeps the best, expressed for the shared update rule as a pseudo-gradient along the improving direction and zero when none improves (Eq. (4)); through Adam this only approximates the direct jump (Appendix A).

Node perturbation (NP). Classic NP [Widrow and Lehr, 1990, Werfel et al., 2003, Fiete and Seung, 2006, Hiratani et al., 2022] injects Gaussian noise $\sigma\boldsymbol{\xi}_l$ into *every* layer's pre-activations on a noisy pass and estimates layer l's gradient as the outer product of the noise-weighted per-example loss change $\delta\mathcal{L}_n\boldsymbol{\xi}_{l,n}/\sigma$ with the clean layer input (Eq. (5)); one clean and 15 noisy passes, averaged.

Activity-based NP (ANP). ANP [Dalm et al., 2026] replaces the injected noise by the measured activity difference $\delta\mathbf{a}_l = \tilde{\mathbf{a}}_l - \mathbf{a}_l$, normalised by the squared activity perturbation summed over all layers and scaled by the total number c of perturbed units (Eq. (6)); same $1{+}15$ passes.

1.5-ANP. 1.5-ANP [Chaubard, 2025a] applies 1.5-SPSA's curvature preconditioner to central-difference activity perturbation ($K = 8$ pairs plus a clean pass, 17 passes), per *attribution unit* (an image example or a text token), with $\alpha = 0.5$ and $\lambda = 10^{-3}$ (Eq. (7)); $\alpha = 0$ gives central-difference ANP, the control that separates attribution from preconditioning (own recipe: Section 5.4).

Iterative NP (INP). INP [Dalm et al., 2026] perturbs *one* layer per pass, so the loss change is attributable to it: a directional derivative along the noise, $\lfloor 15/L \rfloor$ draws per layer (Eq. (8)).

Decorrelated ANP (DANP). DANP [Dalm et al., 2026] is ANP on layer inputs decorrelated by a matrix $\mathbf{R}_l$ trained by the local rule of Ahmad et al. [2023] after each Adam step (Eq. (9)).

Forward-forward (FF). FF [Hinton, 2022] trains each block with a local goodness objective (mean of squares and threshold $\theta_{\text{FF}} = 2$ of the re-implementation of Pezeshki [2023]) on positive and negative data, forwarded as one batch with the neutral-label inputs (three N-sized passes per step on the image tasks, two on the text tasks), and never propagates an error between blocks; we report Hinton's label accumulation and a linear readout on the top block (Appendix A).

Backpropagation (reference) and forward-pass budget. BP feeds the exact gradient of one forward and one backward pass to the same Adam update. The probe methods thus spend 16 forward passes per step, the two 1.5-order rules 17, FF three or two and BP one forward plus one backward (Table 2), so steps per hour differ between methods (Appendix F), and the node-perturbation family obtains one loss difference *per example* whereas SPSA and RS obtain one batch-mean loss per pass: an asymmetry intrinsic to node perturbation that the equal-pass budget compares.

4 Experimental Protocol

Tasks and architectures. An MLP on MNIST [LeCun et al., 1998], a ConvNet on CIFAR-10 [Krizhevsky, 2009], and a character-level LSTM [Hochreiter and Schmidhuber, 1997] and Transformer [Vaswani et al., 2017] on Penn Treebank [Marcus et al., 1993, Mikolov, 2012] ($V = 50$ symbols; 5.10M / 0.40M / 0.45M characters in train / valid / test), all with ≈40k parameters (Table 3, Appendix B). Weights are Xavier-uniform [Glorot and Bengio, 2010] with zero biases (LSTM forget-gate bias 1), and the Transformer's layer normalisation [Ba et al., 2016] has no affine parameters, so every parameter lives in a perturbable layer; the last 5k training images are validation, batches are 128 images or 64 sequences of 64 characters, and there is no augmentation, dropout or weight decay.

Stage 1: ε screening. Every (task, method) pair is run once at $\varepsilon \in \{0.1, 0.01, 0.001\}$ ($\eta = \sigma = \varepsilon$, fixed throughout) for one hour of training on one NVIDIA RTX 5090, with validation and test metrics at initialisation, every 5 minutes of training and at the end (evaluation time excluded); the two 1.5-order rules were added later under the stage-2 settings. Each pair selects the ε with the lowest validation loss at any evaluation (FF on the image tasks: the highest label-accumulation validation accuracy). Because 25 of the 32 original pairs selected this grid's lower edge, $\varepsilon = 10^{-4}$ was added for every pair under the stage-2 settings, and because that value won for eight methods on both image tasks, $\varepsilon = 10^{-5}$ for those sixteen pairs, so that a selected value has a worse neighbour on each side wherever the grid allows (Table 13; five selections sit at a grid edge). On the sequence tasks 10^{-4} loses to 10^{-3} for every rule on the LSTM and for six of the ten on the Transformer, where NP, ANP, 1.5-ANP and FF select 10^{-4} (and lose again at 10^{-5}).

Stage 2: replication. The selected configuration of every pair is run with three seeds (initialisation, data order and probes), evaluated every **minute** of training time, in full fp32 arithmetic (TF32 disabled; Section 5.6 measures its effect), with checkpoints at the best validation loss. A run is stopped and marked diverged when a loss is non-finite or when its clean validation loss exceeds ten times its initial value at two consecutive evaluations, which stops collapsed networks; it also stops over-confident backpropagation runs at $\eta = 10^{-3}$ on CIFAR-10 and 10^{-2} on MNIST (Table 5), none of which is the selected reference (Appendix B: the replaced stage-1 rule). For BP, which has no probe and takes one forward and one backward pass per step (so six to ten times more steps per hour than the probe rules), the reference on each task is the learning rate with the lowest validation loss among $\eta \in \{10^{-2}, \ldots, 10^{-5}\}$ (10^{-5} on the image tasks), selected by the same rule as the probe methods and replicated with three seeds; decoupled weight decay 10^{-2} [Loshchilov and Hutter, 2019] is run as a single-seed check at $\eta \in \{10^{-3}, 10^{-4}\}$ (Section 5.2).

Reporting. The primary metric is the *best validation checkpoint*: per pair and seed, the evaluation with the lowest validation loss over the hour, whose test loss and accuracy or bits per character (loss$/\ln 2$) are reported as the seed mean with the standard deviation in units of the last digit. The end-of-hour value measures stability; FF on the image tasks is selected and reported by label-accumulation accuracy; test data are never used for selection (Appendix B).

Ablations. (i) *Untied probe scale and step size*: the 3×3 grid $\eta, \sigma \in \{10^{-2}, 10^{-3}, 10^{-4}\}$ plus $\sigma = 0.1$ at $\eta = 10^{-4}$ per probe method and task (its $\eta = 10^{-3}$ off-diagonal for the 1.5-order rules only), screened for 30 minutes, with the best untied setting replicated for an hour with three seeds where it beats the best tied value by at least 3 % at the 30-minute checkpoint. (ii) *The 1.5-order rules under their own recipe*: plain SGD, fp64 losses, their own (η, σ) grids and $\alpha = 0$ controls (Section 5.4). (iii) *Controls under the protocol*: 1.5-ANP at the selected step size ($\eta = 10^{-4}$, Adam) with $\sigma \in \{10^{-2}, 10^{-3}\}$ decoupled, fp64 losses and $\alpha \in \{0, 0.25, 0.5, 1\}$ (Table 8), 1.5-SPSA with a non-inert factor. (iv) *Estimator alignment*: the cosine between every estimator and the true gradient of a fixed batch against the number of forward passes (Section 5.6). (v) *Resources*: steps per hour, seconds per step and peak GPU memory of every run (Appendix F). Appendix B specifies each in full and lists hardware, software, code commits and logging.

5 Results

5.1 Main comparison

Table 1 reports the best-validation-checkpoint test loss and accuracy or bits per character of every method on every task, sorted by an overall score (the sum of its four test losses, charted in Figure 1), Table 16 the same runs at the end of the hour and Table 13 the screening behind every selection. On the image tasks the node-perturbation family leads and three of its members tie within seed noise: NP, INP and ANP are within 0.006 of each other on MNIST (0.106–0.112, 96.6–96.9 %) and within 0.05 on CIFAR-10 (1.011–1.058, 63–65 %, seed spread up to 0.06); DANP is 0.09 behind there. The weight-space rules follow at a distance that grows with the task: SPSA and 1.5-SPSA reach 95.8–95.9 % on MNIST and 56–58 % on CIFAR-10, RS 94.9 and 53.7 %. On the sequence tasks INP is the best probe method on the LSTM (2.91 bits per character) and within 0.03 of 1.5-SPSA on the Transformer (3.14 against 3.11), 1.5-SPSA and NP follow on the LSTM (2.96 and 3.02) and SPSA on the Transformer (3.17), and ANP is last on both (4.16 and 3.95). Across the four tasks INP has the best mean rank among the eight probe methods at the best checkpoint (1.5, bootstrap interval 1.25–2.5), ahead of NP (2.75), 1.5-SPSA (3.25), SPSA (4.25), ANP (5.5), RS (6.0), 1.5-ANP (6.25) and DANP (6.5). Against backpropagation at its own best learning rate (97.0 and 69.2 %, Section 5.2) the best probe rule is 0.1 points behind on MNIST and 4 points behind on CIFAR-10, and on the LSTM and the Transformer the gap is 1.1 and 1.3 bits per character, a factor 1.6–1.7 in test loss. The end-of-hour view (Table 16) promotes NP and 1.5-SPSA, which drift least, and on the sequence tasks changes only ANP, 1.5-ANP and DANP, which drift. Figure 2 shows the curves against time at all three sizes (Appendix C: seed bands at 40k and the equal-pass view): on the image tasks BP peaks within five minutes and then grows over-confident, the weight-space rules and 1.5-SPSA have their best checkpoint in the last fifteen minutes, and on the sequence tasks every rule but ANP, 1.5-ANP and DANP is still descending at minute 60.

5.2 The backpropagation reference

With Adam and no schedule, backpropagation completes 2.7–4.3M steps on MNIST (seed spread) and 2.0M on CIFAR-10 in the hour and overfits within minutes: at $\eta = 10^{-3}$ its CIFAR-10 validation loss never returns below its initial value, and at the selected reference ($\eta = 10^{-5}$ on MNIST, 10^{-4} on CIFAR-10) the best checkpoint sits at minute 1–5 (97.0 and 69.2 %) and the hour ends at 96.7 and 65.6 % (Table 5). On the sequence tasks $\eta = 10^{-3}$ is best and BP is still improving at minute

Table 1: Main result, rows sorted by score = the sum of the four test losses. Per task: best-validation-checkpoint test loss and accuracy (%) or bits per character, mean ± s.d. over three seeds (two for the DANP and FF Transformer cells). Bold: best probe method per task. BP (one forward and one backward pass per step) is the reference; FF is reported by label-accumulation accuracy [readout in brackets] and its readout loss is not comparable, so it has no score.

	MNIST / MLP		CIFAR-10 / ConvNet		PTB / char-LSTM		PTB / char-Transformer		
Method	loss	acc. (%)	loss	acc. (%)	loss	bits/char	loss	bits/char	score
BP	0.102±0.004	97.0±0.2	0.907±0.010	69.2±0.6	1.245±0.012	1.80±0.02	1.275±0.014	1.84±0.02	3.53
INP	**0.107±0.004**	**96.9±0.2**	1.055±0.015	63.5±0.5	**2.017±0.055**	**2.91±0.08**	2.176±0.022	3.14±0.03	5.36
NP	0.107±0.002	96.7±0.1	**1.011±0.061**	**65.0±2.6**	2.092±0.067	3.02±0.10	2.328±0.011	3.36±0.02	5.54
1.5-SPSA	0.140±0.008	95.9±0.2	1.215±0.048	56.7±1.9	2.054±0.008	2.96±0.01	**2.155±0.026**	**3.11±0.04**	5.56
SPSA	0.142±0.005	95.8±0.2	1.200±0.051	57.8±2.0	2.141±0.082	3.09±0.12	2.200±0.017	3.17±0.02	5.68
RS	0.168±0.012	94.9±0.3	1.292±0.056	53.7±2.2	2.229±0.092	3.22±0.13	2.276±0.027	3.28±0.04	5.97
1.5-ANP	0.159±0.007	95.7±0.4	1.532±0.041	46.2±1.0	2.167±0.021	3.13±0.03	2.566±0.085	3.70±0.12	6.42
DANP	0.211±0.002	94.0±0.1	1.101±0.018	63.0±1.1	2.635±0.040	3.80±0.06	2.578±0.092	3.72±0.13	6.53
ANP	0.112±0.003	96.6±0.1	1.058±0.025	63.3±1.0	2.885±0.082	4.16±0.12	2.735±0.039	3.95±0.06	6.79
FF	2.335±0.058	59.7±29.5 [11.0]	2.007±0.036	33.9±0.3 [26.9]	2.618±0.035	3.78±0.05	2.932±0.016	4.23±0.02	–

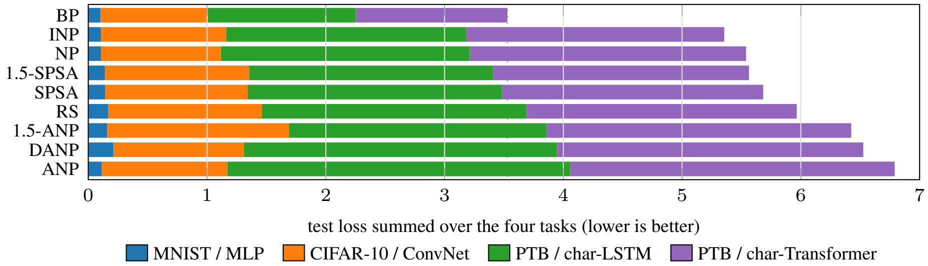

Figure 1: The score of Table 1 as a chart: best-checkpoint test loss summed over the four tasks, with the per-task losses stacked (dotted line: BP); methods ordered from best to worst. FF is omitted (its readout loss is not comparable).

60 (1.80 and 1.84 bits per character). Decoupled weight decay (single seed) adds 0.3–0.5 points on MNIST and removes its drift, and changes little elsewhere (Table 5).

5.3 Probe scale and step size

The tied screening (Table 13) was decisive but for the wrong knob: $\varepsilon \geq 0.01$ collapses most probe runs, 10^{-3} loses none, and 10^{-4} improves eight of the ten methods on both image tasks once more (SPSA from 0.285 to 0.132 in validation loss on MNIST, 1.62 to 1.27 on CIFAR-10), while 10^{-5} is worse for all but four pairs (BP, NP and ANP on MNIST, 1.5-ANP on CIFAR-10, where it ties or wins by a margin within seed noise). The untied grid (Table 6, Appendix C) attributes most of this to the step size: at fixed $\eta = 10^{-4}$ the probe scale ($\sigma \leq 10^{-2}$) changes the 30-minute loss of SPSA, 1.5-SPSA, RS, NP and INP by at most 0.03 on MNIST and 0.09 on CIFAR-10, while ANP and DANP are σ-sensitive on CIFAR-10 (0.14 and 0.21) and 1.5-ANP everywhere; on the image tasks $\eta = 10^{-2}$ loses to 10^{-4} for every rule but DANP on CIFAR-10. A noisy estimate wants a small Adam step and many of them. On the sequence tasks $\eta = 10^{-4}$ loses to 10^{-3} for every rule on the LSTM (12–34k steps per hour) and for seven of ten on the Transformer. The best untied cell beats the best tied one by more than 3 % only for 1.5-ANP on both image tasks (24 and 17 %, at $\sigma \geq 10^{-3}$), DANP on the Transformer (6 %) and ANP on MNIST (4 %); the one-hour replications confirm the 1.5-ANP gains (Table 9).

5.4 The 1.5-order rules under their own recipe

Table 4 (Appendix C) runs both rules as published—plain SGD, fp64 losses, their own (η, σ) and probe-count grids—and Table 8 (Appendix C) runs 1.5-ANP under the protocol's optimizer at the step size the screening selected, with its probe scale freed. Neither recipe beats the protocol: 1.5-SPSA with SGD and 64 pairs per step reaches 0.176 on MNIST and 1.336 on CIFAR-10 against 0.160 and 1.313 for its Adam run at the same 30 minutes, and its $\alpha = 0$ control (plain SPSA with SGD) is better

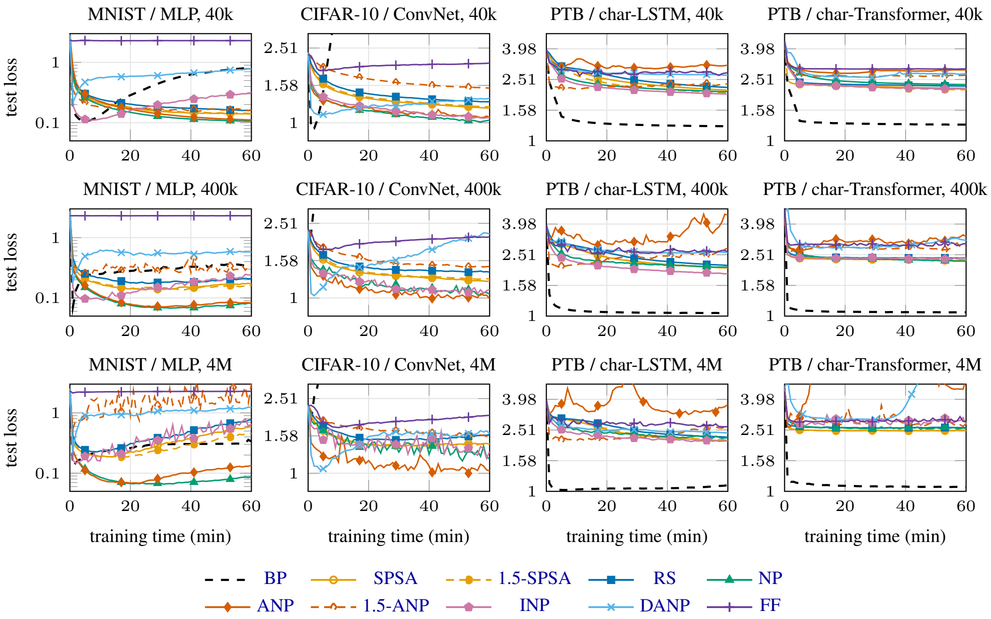

Figure 2: Test loss against training time for every optimizer on every task at the three model sizes (40k: seed mean; 400k and 4M: seed 0; ε selected at 40k). Log y-axis; the 1.5-order rules are dashed in their parent's colour.

by 0.01 on MNIST and 0.11 on the LSTM and equal within 0.01 elsewhere (0.162 versus 0.176, 1.344 versus 1.336, 2.845 versus 2.955, 2.259 versus 2.258). The curvature factor of 1.5-SPSA is thus inert under Adam, which removes its nearly common scale, and slightly harmful under SGD. For 1.5-ANP, Adam at $\eta = 10^{-4}$ with fp64 losses and $\sigma = 10^{-2}$ improves 0.159 to 0.132 on MNIST and 1.53 to 1.28 on CIFAR-10 (its tied selection was limited by a probe scale too small for a second difference in fp32), but at $\sigma = 10^{-3}$ its $\alpha = 0$ control is better still (0.112 and 1.120 against 0.133 and 1.307 with $\alpha = 0.5$), equals ANP on MNIST and comes within 0.06 of it on CIFAR-10; $\alpha = 0.25$ lies in between and $\alpha = 1$ is harmful (0.204 and 1.98). The same holds at the tied $\varepsilon = 10^{-3}$ (Table 7), on the sequence models (Table 8) and, under its own SGD recipe, on two of the four tasks (Table 4). What makes 1.5-ANP usable on the sequence models, where it beats ANP by 1.0 and 0.25 bits per character, is therefore not the curvature exponent; its other differences to ANP (central differences, per-unit normalisation, token-local credit assignment) are not separated by any run here. A curvature term can only help where its estimate is less noisy than the gradient estimate it rescales, which at 8 pairs per step it is not (Table 12 tests 32).

5.5 Model size

Figure 3 repeats the protocol at ten and a hundred times the parameter count (widths scaled, depth unchanged; Table 10): every (task, method) pair reruns for one hour, one seed, at the ε selected at 40k parameters. Three things happen with size (summed loss in Figure 3). At 400k every rule that works at 40k holds or improves (NP 5.54 to 5.47, INP 5.36 to 5.40, the weight-space rules within 0.2), NP reaching 98.1 % on MNIST and the node rules 0.1–0.2 bits per character better on the LSTM. At 4M, with the step size still the one selected at 40k, every probe rule is worse than at 40k, and the order changes: NP is now the best probe rule (5.99) because it keeps improving on MNIST (0.067) while INP, whose $\lfloor 15/L \rfloor$ draws per layer do not grow with width, falls to 6.15, and the weight-space rules lose 0.5–0.6 (SPSA 6.20, RS 6.58), the $\sqrt{K/P}$ penalty of a hundred times more weights. 1.5-ANP's MNIST loss degrades fastest (0.16 to 0.53). BP's summed loss is inflated at both larger sizes by its CIFAR-10 runs, whose validation loss never returns below its initial value at $\eta = 10^{-4}$ (the "best checkpoint" is near step 0); on the other three tasks BP improves with size. With one seed and a step size tuned at 40k, the uniform drop at 4M says that larger models want a smaller η at this budget, not that the rules stop working. Figure 6 shows the per-task curves, and Tables 11 and 12 the follow-up

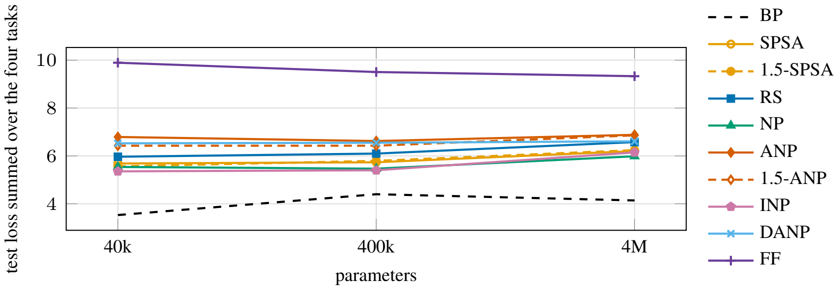

Figure 3: Test loss summed over the four tasks against model size (log scale), one seed per point at the ε selected at 40k parameters; the 40k points are the seed means of Table 1.

runs: random search with direct jumps at the three sizes, and 1.5-ANP with four times more pairs against ANP and INP at the same 65 passes.

5.6 Estimator quality

Figure 7 (Appendix C) measures each estimator's cosine with the true gradient against the number of forward passes: on the MLP the node rules get 0.4–0.6 from the protocol's 16 passes and the weight-space rules 0.01; on the ConvNet every estimator is below 0.1 at 16 passes; on the sequence models none exceeds 0.26 at 1024. 1.5-SPSA is indistinguishable from SPSA, and only 1.5-ANP's second difference degrades as σ shrinks in fp32 (Table 21), which is why it needs $\sigma \geq 10^{-3}$.

5.7 Analysis

Which zero-order method wins? INP, if a single rule must be chosen: the best probe method on the LSTM, within 0.03 bits of the best on the Transformer, within 0.001 of the best on MNIST and within 0.05 on CIFAR-10, and the best mean rank at the best checkpoint (NP has it at the end of the hour). ANP, NP and INP are indistinguishable at the best checkpoint on the image tasks (at the end of the hour ANP drifts least) and DANP is the fastest there; on the sequence tasks INP leads, the weight-space rules and NP follow, 1.5-ANP and DANP trail, and ANP is last. The gap to backpropagation is 0.1 points on MNIST, 4 points on CIFAR-10 and 1.1–1.3 bits per character on the sequence tasks.

Weight probes versus node probes. SPSA and RS perturb all $P \approx 40\text{k}$ weights and obtain one scalar loss difference per probe, so their 16 passes yield a direction with cosine 0.01 to the gradient on the MLP (Figure 7), whereas the node rules obtain a loss difference per example and assign credit per unit (cosine 0.4–0.6); at the right step size the gap is 1 point on MNIST and 7 points on CIFAR-10 at the best checkpoint, and it closes on the sequence models, where SPSA ends within 0.18 and 0.03 bits per character of INP and ahead of NP on the Transformer, as Züge et al. [2023] found for temporally extended tasks.

Curvature preconditioning. 1.5-SPSA reproduces SPSA to within 0.05 in loss on three tasks and is 0.09 better on the LSTM, where SPSA's seed spread is 0.08, at the price of the clean pass (13–15 % fewer steps per hour on the image tasks; on the sequence models its runs completed more steps than SPSA's, a host effect, since both spend 16–17 passes per step): with $\alpha = 0.1$ and $\lambda = 1$ the per-direction factor is nearly constant and Adam removes the remaining common scalar, and Section 5.4 shows the same under plain SGD. 1.5-ANP is one point behind ANP on MNIST and 17 points behind on CIFAR-10 under the tied protocol, because a second difference at $\sigma \leq 10^{-4}$ sits at the fp32 rounding floor; with $\sigma = 10^{-2}$ and fp64 losses it recovers to within 0.03 of ANP on MNIST, and its $\alpha = 0$ control matches ANP on MNIST and comes within 0.06 on CIFAR-10. On the sequence models its token-local attribution repairs ANP's failure (3.13 versus 4.16 bits per character on the LSTM, 3.70 versus 3.95 on the Transformer) without reaching INP or SPSA; per step it is within 3 % of ANP on the ConvNet and 14–43 % faster on the other three models (Table 19).

Activity-based, iterative and decorrelated variants. The measured activity difference (ANP) is worth nothing over NP at the right step size (within seed noise on both image tasks), but on the recurrent and attention models its normalisation by a squared activity difference summed over 25k–48k unit-positions, whose perturbations propagate through shared weights, produces updates of erratic size (the spikes in Figure 4) and the worst probe result on both tasks. INP's one-layer-at-a-time attribution has only $\lfloor 15/L \rfloor$ draws per layer, yet removes all cross-layer noise and is never worse than third on any task. Decorrelation helps only where Dalm et al. [2026] reported it: DANP is the fastest rule on CIFAR-10 (best checkpoint at minute 5, 63.0 %) but drifts to 1.36 by minute 60, is behind on MNIST (94.0 %) and on the sequence tasks (3.80 and 3.72 bits per character), and was the screening's most fragile rule.

Sequence models and forward-forward. Against count models on the training text (unigram 4.32 bits/char, bigram 3.31, trigram 2.61; Appendix G) the zero-order rules learn roughly bigram statistics in an hour—six rules beat the bigram on the LSTM (2.91–3.22) and four on the Transformer (3.11–3.28), none the trigram—and BP (1.80 and 1.84) a real character model; every probe curve is still descending at minute 60, so part of the gap is budget rather than estimator, but at 12–34k steps per hour on the LSTM closing it would take days. FF, not a probe method, trains fast but its two inference rules disagree: on MNIST its selected $\varepsilon = 0.1$ is seed-fragile (label accumulation 59.7(295) % over three seeds against 93.1(3) % at $\varepsilon = 0.01$) and the linear readout collapses to 11–14 %; on CIFAR-10 both numbers are 27–34 %, and on the sequence tasks FF is third-worst on the LSTM and worst on the Transformer (3.78 and 4.23 bits per character; Appendix C).

6 Discussion and Limitations

What the final-loss protocol measures. Under a fixed Adam step with no schedule every probe run at $\varepsilon \geq 10^{-3}$ peaks within minutes and then drifts: once the estimate's signal is exhausted the parameters random-walk with step η and the network grows over-confident. The loss after a fixed hour therefore ranks methods by where their random walk settles, and selecting ε by it produced degenerate choices in stage 1; hence stage 2 reports the best validation checkpoint as primary and the final value for stability. A learning-rate schedule, not in the protocol, would be the other remedy.

Screening, budget and scale. The selection rests on one seed per ε, so the error bars are conditional on ε^* and exclude selection noise. The budget is identical for all methods and the models are small; Section 5.5 shows the weight-space rules losing ground with P. Wall-clock comparisons reward implementations that batch probes well and depend on the host (eight runs at a time here); the equal-pass view (Figure 5) is the implementation-independent comparison.

Faithfulness. Every departure of an implementation from its source (Gaussian rather than Rademacher directions, RS through Adam, FF at the parameter budget, DANP's fixed decorrelation rate) is stated where the method is defined and discussed in Appendix C.

7 Conclusion

We compared nine backprop-free training rules and backpropagation on four ≈40k-parameter architectures under one equal-budget protocol. The screening sweep's first result is methodological: with Adam the step size, not the probe scale, decides how far a noisy estimator gets in an hour, and at $\eta = 10^{-4}$ eight of the ten rules improve and the weight-space rules close about half of their gap on the image tasks. If one zero-order rule must be chosen it is iterative node perturbation, best on the LSTM, within 0.03 bits of the best on the Transformer and within seed noise of the best on the image tasks; classic and activity-based node perturbation match it on images and the activity-based rule fails on sequences; weight-space perturbation competes where the number of perturbed units approaches the number of weights; curvature preconditioning adds nothing to SPSA under Adam or SGD, and the sequence-model gains of 1.5-ANP over ANP survive setting its curvature exponent to zero. Per forward pass the node rules reach backpropagation's best MNIST loss with about ten times more passes and never reach its CIFAR-10 loss; per unit of time the best probe rule is 0.1 and 4 points behind, and on the language models every zero-order rule is still about one bit per character behind after an hour, every curve still descending. At a hundred times the size the same ordering holds with classic node perturbation on top and weight-space perturbation falling behind. Code, job lists and all run logs are public; adding a rule costs about 30 GPU-hours.

References

Nasir Ahmad. Correlations are ruining your gradient descent. *arXiv preprint arXiv:2407.10780*, 2024.

Nasir Ahmad, Ellen Schrader, and Marcel van Gerven. Constrained parameter inference as a principle for learning. *Transactions on Machine Learning Research*, 2023. arXiv:2203.13203.

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

Sergey Bartunov, Adam Santoro, Blake A. Richards, Luke Marris, Geoffrey E. Hinton, and Timothy P. Lillicrap. Assessing the scalability of biologically-motivated deep learning algorithms and architectures. In *Advances in Neural Information Processing Systems*, volume 31, pages 9368–9378, 2018.

Atılım Güneş Baydin, Barak A. Pearlmutter, Don Syme, Frank Wood, and Philip Torr. Gradients without backpropagation. *arXiv preprint arXiv:2202.08587*, 2022.

Gert Cauwenberghs. A fast stochastic error-descent algorithm for supervised learning and optimization. In *Advances in Neural Information Processing Systems*, volume 5, pages 244–251. Morgan Kaufmann, 1992.

Francois Chaubard. 1.5-ANP: Curvature-preconditioned activity perturbation for constant-memory recurrent and token-local pretraining. Manuscript, 2025a.

Francois Chaubard. 1.5-SPSA for large language models. GitHub repository, 2025b. `https://github.com/Fchaubard/1.5-SPSA-LLM`.

Aochuan Chen, Yimeng Zhang, Jinghan Jia, James Diffenderfer, Jiancheng Liu, Konstantinos Parasyris, Yihua Zhang, Zheng Zhang, Bhavya Kailkhura, and Sijia Liu. DeepZero: Scaling up zeroth-order optimization for deep model training. In *International Conference on Learning Representations (ICLR)*, 2024.

Xiangyi Chen, Sijia Liu, Kaidi Xu, Xingguo Li, Xue Lin, Mingyi Hong, and David Cox. ZO-AdaMM: Zeroth-order adaptive momentum method for black-box optimization. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

David G. Clark, L. F. Abbott, and SueYeon Chung. Credit assignment through broadcasting a global error vector. In *Advances in Neural Information Processing Systems*, volume 34, pages 10053–10066, 2021.

Sander Dalm, Joshua Offergeld, Nasir Ahmad, and Marcel van Gerven. Efficient deep learning with decorrelated backpropagation. *arXiv preprint arXiv:2405.02385*, 2024.

Sander Dalm, Marcel van Gerven, and Nasir Ahmad. Node perturbation can effectively train multilayer neural networks. *Neuromorphic Computing and Engineering*, 6(3):034020, 2026. doi: 10.1088/2634-4386/ae9be4. arXiv:2310.00965.

John C. Duchi, Michael I. Jordan, Martin J. Wainwright, and Andre Wibisono. Optimal rates for zero-order convex optimization: The power of two function evaluations. *IEEE Transactions on Information Theory*, 61(5):2788–2806, 2015. doi: 10.1109/TIT.2015.2409256.

Ila R. Fiete and H. Sebastian Seung. Gradient learning in spiking neural networks by dynamic perturbation of conductances. *Physical Review Letters*, 97(4):048104, 2006. doi: 10.1103/PhysRevLett.97.048104.

Abraham D. Flaxman, Adam Tauman Kalai, and H. Brendan McMahan. Online convex optimization in the bandit setting: Gradient descent without a gradient. In *Proceedings of the Sixteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 385–394, 2005.

Barry Flower and Marwan Jabri. Summed weight neuron perturbation: An O(N) improvement over weight perturbation. In *Advances in Neural Information Processing Systems*, volume 5, pages 212–219. Morgan Kaufmann, 1992.

Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, volume 9 of *Proceedings of Machine Learning Research*, pages 249–256, 2010.

Nikolaus Hansen. The CMA evolution strategy: A tutorial. *arXiv preprint arXiv:1604.00772*, 2016.

Nikolaus Hansen, Sibylle D. Müller, and Petros Koumoutsakos. Reducing the time complexity of the derandomized evolution strategy with covariance matrix adaptation (CMA-ES). *Evolutionary Computation*, 11(1):1–18, 2003. doi: 10.1162/106365603321828970.

Geoffrey Hinton. The forward-forward algorithm: Some preliminary investigations. *arXiv preprint arXiv:2212.13345*, 2022.

Naoki Hiratani, Yash Mehta, Timothy Lillicrap, and Peter E. Latham. On the stability and scalability of node perturbation learning. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8): 1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.

Marwan Jabri and Barry Flower. Weight perturbation: An optimal architecture and learning technique for analog VLSI feedforward and recurrent multilayer networks. *IEEE Transactions on Neural Networks*, 3(1):154–157, 1992. doi: 10.1109/72.105429.

Max Jaderberg, Wojciech Marian Czarnecki, Simon Osindero, Oriol Vinyals, Alex Graves, David Silver, and Koray Kavukcuoglu. Decoupled neural interfaces using synthetic gradients. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1627–1635, 2017.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

Julien Launay, Iacopo Poli, François Boniface, and Florent Krzakala. Direct feedback alignment scales to modern deep learning tasks and architectures. In *Advances in Neural Information Processing Systems*, volume 33, pages 9346–9360, 2020.

Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.

Karel Lenc, Erich Elsen, Tom Schaul, and Karen Simonyan. Non-differentiable supervised learning with evolution strategies and hybrid methods. *arXiv preprint arXiv:1906.03139*, 2019.

Timothy P. Lillicrap, Daniel Cownden, Douglas B. Tweed, and Colin J. Akerman. Random synaptic feedback weights support error backpropagation for deep learning. *Nature Communications*, 7: 13276, 2016. doi: 10.1038/ncomms13276.

Sijia Liu, Pin-Yu Chen, Bhavya Kailkhura, Gaoyuan Zhang, Alfred O. Hero, and Pramod K. Varshney. A primer on zeroth-order optimization in signal processing and machine learning: Principals, recent advances, and applications. *IEEE Signal Processing Magazine*, 37(5):43–54, 2020. doi: 10.1109/MSP.2020.3003837.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1711.05101.

Sadhika Malladi, Tianyu Gao, Eshaan Nichani, Alex Damian, Jason D. Lee, Danqi Chen, and Sanjeev Arora. Fine-tuning language models with just forward passes. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Horia Mania, Aurelia Guy, and Benjamin Recht. Simple random search of static linear policies is competitive for reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

Mitchell P. Marcus, Beatrice Santorini, and Mary Ann Marcinkiewicz. Building a large annotated corpus of English: The Penn Treebank. *Computational Linguistics*, 19(2):313–330, 1993.

J. Matyas. Random optimization. *Automation and Remote Control*, 26(2):246–253, 1965.

Tomáš Mikolov. *Statistical language models based on neural networks*. PhD thesis, Brno University of Technology, Faculty of Information Technology, 2012. `https://www.fit.vut.cz/study/phd-thesis/283/`.

Yurii Nesterov and Vladimir Spokoiny. Random gradient-free minimization of convex functions. *Foundations of Computational Mathematics*, 17(2):527–566, 2017. doi: 10.1007/s10208-015-9296-2.

Arild Nøkland. Direct feedback alignment provides learning in deep neural networks. In *Advances in Neural Information Processing Systems*, volume 29, 2016.

Mohammad Pezeshki. pytorch_forward_forward: a reimplementation of Hinton's forward-forward algorithm. GitHub repository, `https://github.com/mohammadpz/pytorch_forward_forward`, 2023.

Leonard A. Rastrigin. The convergence of the random search method in the extremal control of a many parameter system. *Automation and Remote Control*, 24(10):1337–1342, 1963.

Mengye Ren, Simon Kornblith, Renjie Liao, and Geoffrey Hinton. Scaling forward gradient with local losses. In *International Conference on Learning Representations (ICLR)*, 2023.

David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. doi: 10.1038/323533a0.

Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint arXiv:1703.03864*, 2017.

Albert Jiménez Sanfiz and Mohamed Akrout. Benchmarking the accuracy and robustness of feedback alignment algorithms. *arXiv preprint arXiv:2108.13446*, 2021.

M. A. Schumer and Kenneth Steiglitz. Adaptive step size random search. *IEEE Transactions on Automatic Control*, 13(3):270–276, 1968. doi: 10.1109/TAC.1968.1098903.

David Silver, Anirudh Goyal, Ivo Danihelka, Matteo Hessel, and Hado van Hasselt. Learning by directional gradient descent. In *International Conference on Learning Representations (ICLR)*, 2022.

James C. Spall. Multivariate stochastic approximation using a simultaneous perturbation gradient approximation. *IEEE Transactions on Automatic Control*, 37(3):332–341, 1992. doi: 10.1109/9.119632.

James C. Spall. An overview of the simultaneous perturbation method for efficient optimization. *Johns Hopkins APL Technical Digest*, 19(4):482–492, 1998.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Justin Werfel, Xiaohui Xie, and H. Sebastian Seung. Learning curves for stochastic gradient descent in linear feedforward networks. In *Advances in Neural Information Processing Systems*, volume 16, pages 1197–1204, 2003.

Justin Werfel, Xiaohui Xie, and H. Sebastian Seung. Learning curves for stochastic gradient descent in linear feedforward networks. *Neural Computation*, 17(12):2699–2718, 2005. doi: 10.1162/089976605774320539.

Bernard Widrow and Michael A. Lehr. 30 years of adaptive neural networks: Perceptron, Madaline, and backpropagation. *Proceedings of the IEEE*, 78(9):1415–1442, 1990. doi: 10.1109/5.58323.

Daan Wierstra, Tom Schaul, Tobias Glasmachers, Yi Sun, Jan Peters, and Jürgen Schmidhuber. Natural evolution strategies. *Journal of Machine Learning Research*, 15(27):949–980, 2014.

Xingwen Zhang, Jeff Clune, and Kenneth O. Stanley. On the relationship between the OpenAI evolution strategy and stochastic gradient descent. *arXiv preprint arXiv:1712.06564*, 2017.

Yihua Zhang, Pingzhi Li, Junyuan Hong, Jiaxiang Li, Yimeng Zhang, Wenqing Zheng, Pin-Yu Chen, Jason D. Lee, Wotao Yin, Mingyi Hong, Zhangyang Wang, Sijia Liu, and Tianlong Chen. Revisiting zeroth-order optimization for memory-efficient LLM fine-tuning: A benchmark. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 59173–59190, 2024.

Paul Züge, Christian Klos, and Raoul-Martin Memmesheimer. Weight versus node perturbation learning in temporally extended tasks: Weight perturbation often performs similarly or better. *Physical Review X*, 13(2):021006, 2023. doi: 10.1103/PhysRevX.13.021006.

A Optimizer definitions

This appendix gives the shared update rule and the full definition of every rule summarised in Section 3.2; Table 2 lists the forward-pass budget of every method per step.

Shared update rule. Every estimate $\hat{\mathbf{g}}$ is consumed by Adam [Kingma and Ba, 2015] with $(\beta_1, \beta_2) = (0.9, 0.99)$, learning rate η and no schedule:

$$\mathbf{m} \leftarrow \beta_1 \mathbf{m} + (1 - \beta_1)\,\hat{\mathbf{g}}, \qquad \mathbf{v} \leftarrow \beta_2 \mathbf{v} + (1 - \beta_2)\,\hat{\mathbf{g}}^2, \qquad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta\,\frac{\hat{\mathbf{m}}}{\sqrt{\hat{\mathbf{v}}} + 10^{-8}}. \tag{1}$$

Adam's constants. $(\beta_1, \beta_2) = (0.9, 0.99)$ is a protocol choice made before any run (the usual $\beta_2 = 0.999$ would average the second moment over a thousand steps, several seconds of these runs); it is shared by every method and is not tuned.

SPSA (central difference, Gaussian probes). With $K = 8$ antithetic pairs of probes $\mathbf{z}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_P)$,

$$\hat{\mathbf{g}}_{\text{SPSA}} = \frac{1}{K} \sum_{k=1}^{K} \frac{\mathcal{L}(\boldsymbol{\theta} + \sigma \mathbf{z}_k) - \mathcal{L}(\boldsymbol{\theta} - \sigma \mathbf{z}_k)}{2\sigma}\,\mathbf{z}_k, \tag{2}$$

the Gaussian-smoothing estimator of Nesterov and Spokoiny [2017] and the multi-direction form of SPSA [Spall, 1992] with Gaussian rather than Rademacher directions. Each pair costs two forward passes on the batch-mean loss, so the 16-pass budget buys eight directions. Parameters are perturbed in place and restored exactly from a flat copy, as in MeZO [Malladi et al., 2023]; no clean pass is spent, and the training loss logged for SPSA is the mean of the 16 probe losses.

1.5-SPSA. 1.5-SPSA [Chaubard, 2025b] spends one clean pass on $\mathcal{L}_0 = \mathcal{L}(\boldsymbol{\theta})$ and uses the three-point second difference along every direction to damp the step where the loss is sharp:

$$\begin{gathered} \hat{\mathbf{g}}_{1.5\text{-SPSA}} = \frac{1}{K} \sum_{k=1}^{K} \frac{\mathcal{L}(\boldsymbol{\theta} + \sigma \mathbf{z}_k) - \mathcal{L}(\boldsymbol{\theta} - \sigma \mathbf{z}_k)}{2\sigma\, c_k}\,\mathbf{z}_k, \\ c_k = \max\bigl(\hat{\kappa}_k^{\alpha}, \lambda\bigr), \qquad \hat{\kappa}_k = \frac{|\mathcal{L}(\boldsymbol{\theta} + \sigma \mathbf{z}_k) - 2\mathcal{L}_0 + \mathcal{L}(\boldsymbol{\theta} - \sigma \mathbf{z}_k)|}{\sigma^2}, \end{gathered} \tag{3}$$

with the saturating exponent $\alpha = 0.1$ and floor $\lambda = 1$ of the published code. With $K = 8$ pairs the step costs 17 passes. The published method uses Rademacher directions and a plain SGD step $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta\,\hat{\mathbf{g}}$ with $\eta = \sigma$ tied, i.e. $\Delta\boldsymbol{\theta} = -\frac{1}{2K}\sum_k \frac{\Delta\mathcal{L}_k}{c_k}\mathbf{z}_k$, many directions per step and few steps [Chaubard, 2025b]. Under the protocol the estimate goes to Adam like every other method, which makes c_k inert (Section 5.6); Section 5.4 therefore also runs the rule under its own recipe. We use Gaussian directions, like SPSA, throughout.

Random search. Random search [Rastrigin, 1963, Matyas, 1965, Schumer and Steiglitz, 1968] evaluates the clean loss $\mathcal{L}_0$ and $K = 15$ candidates $\boldsymbol{\theta} + \sigma \mathbf{z}_k$ and keeps the best, $k^* = \arg\min_k \mathcal{L}(\boldsymbol{\theta} + \sigma \mathbf{z}_k)$. To fit the shared update rule it is expressed through the pseudo-gradient

$$\hat{\mathbf{g}}_{\text{RS}} = \begin{cases} \dfrac{\mathcal{L}_{k^*} - \mathcal{L}_0}{\sigma}\, \mathbf{z}_{k^*} & \text{if } \mathcal{L}_{k^*} < \mathcal{L}_0, \\ \mathbf{0} & \text{otherwise,} \end{cases} \tag{4}$$

which points along the improving direction and is zero when no candidate improves. Through Adam this is only an approximation of the jump $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \sigma \mathbf{z}_{k^*}$: the first step moves by $\approx \eta \operatorname{sign}(\mathbf{z}_{k^*})$ per coordinate, later steps are bent by momentum, and rejected steps still move the parameters along the decaying momentum; the direct-jump variant ("jumping" random search without Adam) is implemented in the released code but was not run within the budget.

Node perturbation (NP). Classic node perturbation [Widrow and Lehr, 1990, Werfel et al., 2003, Fiete and Seung, 2006, Hiratani et al., 2022] perturbs the units rather than the weights. For layer l with input $\mathbf{h}_{l-1}$, pre-activation $\mathbf{a}_l = \mathbf{W}_l \mathbf{h}_{l-1} + \mathbf{b}_l$ and post-activation $\mathbf{h}_l = f(\mathbf{a}_l)$, a noisy pass injects Gaussian noise into *every* layer and propagates it, $\tilde{\mathbf{a}}_l = \mathbf{W}_l \tilde{\mathbf{h}}_{l-1} + \mathbf{b}_l + \sigma \boldsymbol{\xi}_l$ with $\boldsymbol{\xi}_l \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. Writing $\delta\mathcal{L}_n = \tilde{\ell}_n - \ell_n$ for the change of the per-example loss, the estimator for layer l is

$$\hat{\mathbf{g}}^{\text{NP}}_{\mathbf{W}_l} = \frac{1}{N} \sum_{n=1}^{N} \frac{\delta\mathcal{L}_n}{\sigma}\, \boldsymbol{\xi}_{l,n}\, \mathbf{h}^{\top}_{l-1,n}, \qquad \hat{\mathbf{g}}^{\text{NP}}_{\mathbf{b}_l} = \frac{1}{N} \sum_{n=1}^{N} \frac{\delta\mathcal{L}_n}{\sigma}\, \boldsymbol{\xi}_{l,n}, \tag{5}$$

the outer product of the noise-weighted loss change with the *clean* layer input (for convolutions, the matching correlation over unfolded patches; for embeddings, the one-hot input), averaged over 15 noisy passes. $\delta\mathcal{L}_n \boldsymbol{\xi}_l / \sigma$ equals $\delta\mathcal{L}_n \mathbf{v}_l / \sigma^2$ with $\mathbf{v}_l = \sigma \boldsymbol{\xi}_l$, the form of Dalm et al. [2026]. For a layer applied several times per forward (the LSTM gate layer, once per time step) the same $\delta\mathcal{L}_n$ weights every call.

Activity-based node perturbation (ANP). Because injected noise propagates, the actual perturbation of layer l is the measured activity difference $\delta\mathbf{a}_l = \tilde{\mathbf{a}}_l - \mathbf{a}_l$ rather than $\sigma\boldsymbol{\xi}_l$. Activity-based NP [Dalm et al., 2026] uses this difference and normalises by the norm of the total activity perturbation across all layers:

$$\hat{\mathbf{g}}^{\text{ANP}}_{\mathbf{W}_l} = \frac{1}{N} \sum_{n=1}^{N} c\, \frac{\delta\mathcal{L}_n}{\|\delta\mathbf{a}_n\|^2}\, \delta\mathbf{a}_{l,n}\, \mathbf{h}^{\top}_{l-1,n}, \qquad \|\delta\mathbf{a}_n\|^2 = \sum_{l'=1}^{L} \|\delta\mathbf{a}_{l',n}\|^2, \tag{6}$$

with $c = \sum_l N_l$ the total number of perturbed units (all layers, all time steps or positions; the input is not counted), following the authors' reference implementation. For a single perturbed unit it reduces to Eq. (5).

1.5-ANP. 1.5-ANP [Chaubard, 2025a] combines activity-based perturbation with antithetic pairs and the three-point curvature preconditioner of 1.5-SPSA, applied per *attribution unit* u: an example on the image tasks and a (sequence, position) token on the language-modelling tasks ("token-local" attribution, with the LSTM gate layer called once per position). Each of $K = 8$ pairs injects $\pm\sigma\boldsymbol{\xi}$ into every layer; with $\mathbf{d}_u = \mathbf{a}^+_u - \mathbf{a}^-_u$ the realized central activity difference over the N_u perturbed nodes of u and $\ell^0_u, \ell^{\pm}_u$ the clean and perturbed losses of u,

$$\hat{c}_u = \frac{4N_u\, |\ell^+_u - 2\ell^0_u + \ell^-_u|}{\|\mathbf{d}_u\|^2}, \qquad \beta_u = \frac{N_u\, (\ell^+_u - \ell^-_u)}{\|\mathbf{d}_u\|^2\, (\hat{c}_u + \lambda)^{\alpha}}, \qquad \hat{\mathbf{g}}^{\text{1.5-ANP}}_{\mathbf{a}_u} = \frac{1}{K} \sum_{k=1}^{K} \beta^{(k)}_u \mathbf{d}^{(k)}_u, \tag{7}$$

with $\lambda = 10^{-3}$ (10^{-4} on the LSTM in the specification) and $\alpha = 0.5$; the weight update is the batch mean of $\hat{\mathbf{g}}_{\mathbf{a}_u} \mathbf{h}^{\top}_u$ as in Eq. (6), and N_u counts the perturbed nodes of one unit (90 on the MLP, 402 per LSTM time step, 754 per Transformer token). With $\alpha = 0$ the rule is central-difference ANP, the control that separates attribution from preconditioning. The step costs 17 passes. The specification uses plain SGD with a learning rate 20–50 times σ and damps the deviation of the perturbed recurrent state by one half at every time step; the protocol runs tie $\eta = \sigma$ under Adam without damping, and Section 5.4 runs the rule under its own recipe with both (the settings its own grid selects have η/σ between 0.5 and 5, Table 4, not the specification's 20–50). The three-point second differences that define $\hat{c}_u$ are, at $\sigma = 10^{-3}$, of the order of the fp32 rounding of the loss, so the own-recipe runs evaluate the losses in fp64 (Section 5.6).

Table 2: Forward-pass budget per step. L is the number of perturbable layers (3 for the MLP and the LSTM, 5 for the ConvNet, 15 for the Transformer).

Method	Clean	Probe passes	Backward	Estimate per step
BP	1	0	full	exact gradient
SPSA	0	16	–	8 central differences, averaged
1.5-SPSA	1	16	–	8 curvature-scaled central differences
RS	1	15	–	best of 15 candidates
NP, ANP, DANP	1	15 (all layers)	–	15 noise draws, averaged (DANP: $+\mathbf{R}_l$ update)
1.5-ANP	1	16 (all layers, 8 pairs)	–	8 curvature-scaled central differences per unit
INP	1	15 (one layer each)	–	$\lfloor 15/L \rfloor$ draws per layer
FF	3 / 2	0	per block	local goodness gradients

Iterative node perturbation (INP). Iterative NP [Dalm et al., 2026] perturbs *one* layer per noisy pass, so that the loss change $\delta\mathcal{L}_n^{(l)}$ is attributable to that layer and the estimator is a directional derivative along the injected noise $\mathbf{v}_{l,n} = \sigma\boldsymbol{\xi}_{l,n}$,

$$\hat{\mathbf{g}}^{\text{INP}}_{\mathbf{W}_l} = \frac{1}{N}\sum_{n=1}^{N} N_l \, \frac{\delta\mathcal{L}_n^{(l)}}{\|\mathbf{v}_{l,n}\|^2} \, \mathbf{v}_{l,n} \, \mathbf{h}_{l-1,n}^{\top}, \tag{8}$$

with N_l the number of perturbed units of layer l. The 15 noisy passes give $\lfloor 15/L \rfloor$ draws per layer: 5 for the MLP and the LSTM ($L = 3$), 3 for the ConvNet ($L = 5$) and 1 for the Transformer ($L = 15$). For the LSTM gate layer, called once per time step, the norm in Eq. (8) is taken over the injected noise of all calls, whereas the reference code of Dalm et al. [2026] (feed-forward only) uses the measured activity difference; the two coincide for a layer called once.

Decorrelated ANP (DANP). Decorrelated ANP [Dalm et al., 2026] is ANP in which every layer input is first decorrelated by a square matrix $\mathbf{R}_l$ that is not a parameter of the model, $\hat{\mathbf{h}}_{l-1} = \mathbf{R}_l\mathbf{h}_{l-1}$, with $\mathbf{R}_l$ initialised to the identity and trained by the local rule of Ahmad et al. [2023],

$$\mathbf{R}_l \leftarrow \mathbf{R}_l - \eta_R \left(\hat{\mathbf{C}}_l - \operatorname{diag}\hat{\mathbf{C}}_l\right)\mathbf{R}_l, \qquad \hat{\mathbf{C}}_l = \tfrac{1}{M}\textstyle\sum_{m=1}^{M} \hat{\mathbf{h}}_{l-1,m}\hat{\mathbf{h}}_{l-1,m}^{\top}, \tag{9}$$

from the decorrelated inputs recorded on the clean pass (M rows: examples, or examples × positions / patches). The decorrelation rate is fixed at $\eta_R = 10^{-3}$, the value of Dalm et al. [2026] and the default of their code, and is not tied to ε; $\mathbf{R}_l$ is updated once per step after the Adam step. Embeddings, whose inputs are one-hot, are not decorrelated.

Forward-forward (FF). Forward-forward [Hinton, 2022] trains each block with a local objective and never propagates an error between blocks. With goodness $G = \text{mean}_j\, h_{l,j}^2$ (the mean of squares and threshold $\theta_{\text{FF}} = 2$ of the reference re-implementation of Pezeshki [2023]; Hinton uses the sum of squares), the local loss on a positive/negative pair is $\ell_{\text{FF}} = \log(1 + e^{\theta_{\text{FF}} - G_{\text{pos}}}) + \log(1 + e^{G_{\text{neg}} - \theta_{\text{FF}}})$, and activities are ℓ_2-normalised before the next block. A block is one hidden layer of the MLP, one convolution–ReLU–pool stage or the dense layer of the ConvNet, the LSTM layer, and one pre-LN block of the Transformer; for text the goodness is computed per position. For images, positive data carry the correct label overlaid on the first 10 pixels and negative data a wrong label; for text, negatives have 30 % of their characters replaced at random. The positive, negative and (for images) neutral-label inputs are forwarded as one batch, i.e. three N-sized passes per step on the image tasks and two on the text tasks. Two inference rules are reported: Hinton's label accumulation (the label whose overlay yields the highest total goodness over all blocks but the first, Section 3.3 of Hinton [2022]; we also log the all-blocks variant) and a linear readout trained on the normalised, detached top-block activity of the neutral-label pass (Hinton's softmax reads all hidden layers but the first; reading only the top block keeps the readout inside the parameter budget). The LSTM block's output lies in $(-1, 1)$, so its goodness cannot exceed 1; that block uses $\theta_{\text{FF}} = 0.25$.

Backpropagation (reference). One forward and one backward pass produce the exact gradient, fed to the same Adam update. It is the reference, not a zero-order method.

B Protocol details

Table 3 lists the four tasks of Section 4; the reporting conventions, the divergence rule, the ablations and the hardware and software of the protocol are specified in full below.

Table 3: Tasks, splits (train / validation / test) and architectures. Hidden nonlinearities are ReLU; convolutions are 3×3 with 2×2 max-pooling; the LSTM [Hochreiter and Schmidhuber, 1997] has one layer of 80 units; the Transformer [Vaswani et al., 2017] has two pre-LN blocks of width 48, 4 heads, feed-forward width 64, learned positional embeddings and context $T = 64$. *Units* is the number of pre-activation values one node-perturbation probe perturbs for a single example, summed over layers, time steps and spatial positions (the Transformer's position embedding, 64×48 values, is perturbed once per batch and counted once); it is the normalisation constant c of Eq. (6).

Key	Data	Split	Model	Params	Units
`mlp_mnist`	MNIST	55k / 5k / 10k	MLP 784–48–32–10	39,578	90
`conv_cifar10`	CIFAR-10	45k / 5k / 10k	conv16–conv32–conv32–fc48–10	39,450	26,682
`lstm_ptb`	PTB (char)	standard	emb32–LSTM80–V	41,810	25,728
`transformer_ptb`	PTB (char)	standard	emb48 + pos64–2 blocks–V	39,250	48,256

Reporting conventions. The primary metric is the *best validation checkpoint*: for each pair and seed, the evaluation with the lowest validation loss over the hour, whose test loss, test accuracy (classification) or bits per character (loss/ $\ln 2$, language modelling) is reported as the mean over seeds with the standard deviation in units of the last digit ("0.140(5)" is 0.140 ± 0.005; the sample standard deviation over three seeds; cells without parentheses are single runs). The value at the end of the hour is reported as a secondary stability metric, and for the two classification tasks the checkpoint with the highest validation *accuracy* is reported as well (Appendix F), since loss and accuracy drift apart under a fixed step. FF on the image tasks is selected and reported by label-accumulation accuracy, with the readout accuracy in brackets. Mean ranks are over the eight probe methods, with a 95 % bootstrap interval over seeds. Test data are never used for selection.

Selection with disagreeing criteria. Where the stage-1 selection by best and by final validation loss disagree in a non-degenerate way (DANP on both image tasks), both ε are replicated and the better mean validation loss decides.

Divergence rule. Stage 2 stops a run and marks it diverged when a loss is non-finite or when its clean validation loss exceeds ten times its initial value at two consecutive evaluations. Stage 1 used a factor two and additionally compared each method's own training loss with its first-step value; the former stopped backpropagation on CIFAR-10 at 67 % accuracy and the latter was method-dependent because SPSA's training loss is a probe mean, so both were replaced; a run that the old rule did not stop is identical under the new one.

Ablations. (i) *Untied probe scale and step size*: for every probe method and task the grid $\eta \in \{10^{-2}, 10^{-3}, 10^{-4}\} \times \sigma \in \{10^{-2}, 10^{-3}, 10^{-4}\}$ plus $\sigma = 0.1$ at $\eta = 10^{-4}$, screened for 30 minutes at 1-minute granularity (the tied diagonal is taken from the one-hour runs at minute 30); the best untied setting of a pair is then replicated for one hour with three seeds where it beats the best tied diagonal cell by at least 3 % (at most twelve pairs, largest gains first; Table 9 reports the replications that completed before the deadline), so that a rule is also compared at its own best (η, σ) at equal compute and without any schedule. (ii) *The 1.5-order rules under their own recipe*: plain SGD instead of Adam, fp64 loss evaluation, 1.5-SPSA with $\eta = \sigma \in \{10^{-2}, 3 \cdot 10^{-3}, 10^{-3}, 3 \cdot 10^{-4}, 10^{-4}\}$ and 8 or 64 pairs with the $\alpha = 0$ (1SPSA) control, 1.5-ANP with $\eta \in \{0.05, 0.02, 0.01, 0.005\}$ decoupled from $\sigma \in \{10^{-2}, 10^{-3}\}$ with the $\alpha = 0$ control, a $\lambda = 10^{-2}$ floor (never selected) and, on the LSTM, the recurrent-state damping; 30-minute screening, single seed (Section 5.4). (iii) *Controls under the protocol*: 1.5-ANP with Adam at the step size the screening selected on the image tasks ($\eta = 10^{-4}$), the probe scale decoupled ($\sigma \in \{10^{-2}, 10^{-3}\}$), fp64 losses, the curvature exponent $\alpha \in \{0, 0.25, 0.5, 1\}$ and, on the LSTM, the recurrent-state damping (Table 8); 1.5-ANP with $\alpha = 0$ and with fp64 losses at $\varepsilon = 10^{-3}$; 1.5-SPSA with a non-inert factor ($\alpha = 0.5$, $\lambda = 10^{-3}$). (iv) *Estimator alignment*: the cosine between every estimator and the true gradient of a fixed batch as a function of the number of forward passes, per layer after a 2000-step backpropagation warm-up, in fp32, TF32 and fp64, for σ from 10^{-1} to 10^{-5} (Appendix F). (v) *Resources*: every run records its steps, seconds of training time per step and peak GPU memory (Appendix F); throughput is measured with eight runs sharing one host.

Not run. Three planned ablations did not run before the deadline and are reported nowhere: the isolated 150-second reruns of every selected configuration (throughput and peak memory are instead taken from the stage-2 runs, eight per host), the direct-jump random search, and the $\eta = 10^{-3}$ off-diagonal cells of the untied grid for the six original probe methods; their job lists are in the repository.

Hardware and software. All runs use one RTX 5090 per run on an 8-GPU vast.ai node (8 runs at a time; PyTorch 2.9.1, CUDA 12.8), with the code at `https://github.com/Fchaubard/zero-order-ablations` (stage 1: commits `db100bc` and `f5424e9`; stage 2: `97c03f5`). Every run is logged to Weights & Biases (project `zoo-survey`) and writes a JSON file with its configuration, evaluation history and final metrics.

C Additional results and discussion

Table 4: The 1.5-order rules under their own recipe (plain SGD, fp64 losses, 30-minute screening): the best configuration by validation loss and its best-checkpoint test loss (accuracy or bits per character), the $\alpha = 0$ control at the same grid, and the protocol run (Adam, tied ε) at minute 30. Config key: lr $= \eta$; K17 / K129 = 8 / 64 antithetic pairs per step; sig $= \sigma$; lam $= \lambda$; l64 = fp64 losses.

Task	Rule	best config	test loss (acc / bpc)	$\alpha = 0$ control	protocol (Adam) @30 min
MNIST / MLP	1.5-SPSA	lr0.003 K129 l64 sgd	0.176 (94.8)	0.162 (94.9)	0.160±0.016
	1.5-ANP	lr0.01 sig0.01 lam0.001 l64 sgd	0.137 (96.0)	0.121 (96.6)	0.167±0.015
CIFAR-10 / ConvNet	1.5-SPSA	lr0.003 K129 l64 sgd	1.336 (51.7)	1.344 (51.6)	1.313±0.050
	1.5-ANP	lr0.005 sig0.001 lam0.001 l64 sgd	2.356 (20.5)	2.406 (9.4)	1.634±0.067
PTB / char-LSTM	1.5-SPSA	lr0.01 K17 l64 sgd	2.955 (4.26)	2.845 (4.10)	2.234±0.001
	1.5-ANP	lr0.05 sig0.01 lam0.0001 l64 sgd	2.463 (3.55)	2.424 (3.50)	2.167±0.021
PTB / char-Transformer	1.5-SPSA	lr0.01 K17 l64 sgd	2.258 (3.26)	2.259 (3.26)	2.230±0.004
	1.5-ANP	lr0.005 sig0.01 lam0.001 l64 sgd	2.763 (3.99)	2.763 (3.99)	2.585±0.076

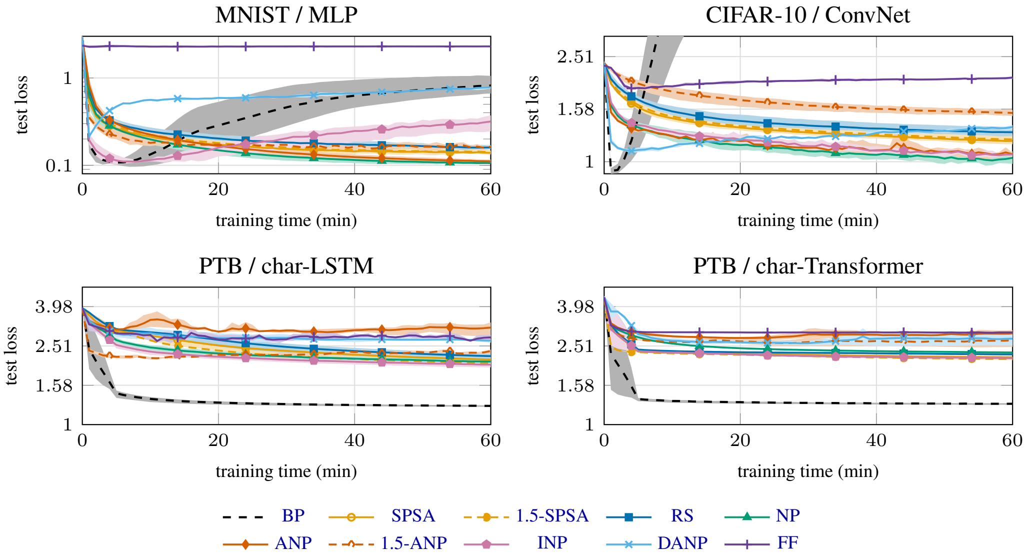

Figure 4: Test loss versus training time (minutes; evaluation excluded) of the selected runs: seed mean (line) and seed range (shaded). Log y-axis. The 1.5-order rules are dashed in their parent's colour.

Equal-pass view. Figure 5 shows the selected runs of Figure 4 against the number of forward passes (best validation checkpoint within the pass budget, mean over seeds), which removes the implementation- and host-dependent throughput of each method from the comparison. Backpropagation reaches 0.139 on MNIST and 0.907 on CIFAR-10 within 10^5 passes, where every probe

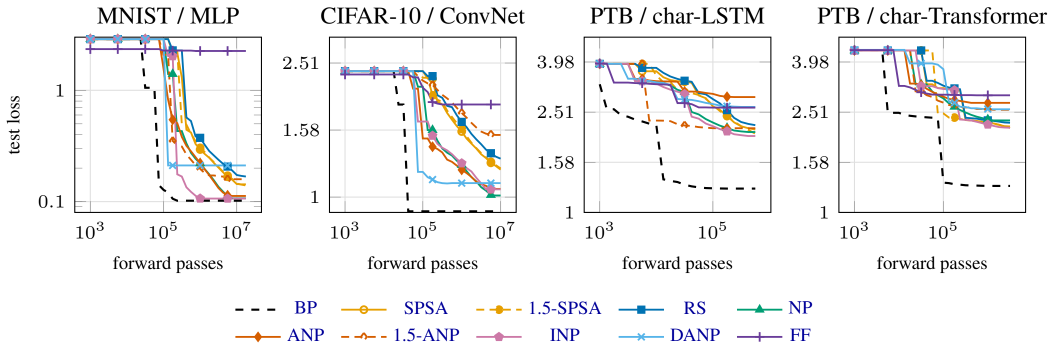

Figure 5: Equal-compute view: test loss versus the number of forward passes (one BP step counts one pass plus its backward), interpolated between the 1-minute evaluations, mean over seeds.

Table 5: Backpropagation reference under the stage-2 protocol: test accuracy (%) or bits per character at the best validation checkpoint (with its minute) and at the end of the hour, for learning rates η and decoupled weight decay (AdamW). The reference of Table 1 is the row with the lowest validation loss on each task (10^{-5} on MNIST, 10^{-4} on CIFAR-10, 10^{-3} on both sequence tasks); the $\eta = 0.01$ image rows are stage-1 runs. A dagger marks a run stopped by the divergence rule (over-confident, not collapsed, in every daggered case).

		MNIST		CIFAR-10		LSTM		Transformer	
lr	decay	best (min)	final	best (min)	final	best (min)	final	best (min)	final
0.01	0	12.7 (0)	67.1†	62.7 (1)	59.3	1.85 (20)	1.93	1.89 (40)	1.90
0.001	0	97.2±0.2 (1)	97.4±0.0	9.4 (0)	66.9†	1.80±0.02 (60)	1.80±0.02	1.84±0.02 (57)	1.84±0.02
0.0001	0	97.2±0.1 (1)	97.1±0.0	69.2±0.6 (1)	65.6±0.3	2.12 (60)	2.12	1.93 (60)	1.93
0.00001	0	97.0±0.2 (5)	96.7±0.2	68.9 (9)	65.6	–	–	–	–
0.001	0.01	97.5 (4)	97.9	9.4 (0)	67.4	1.78 (60)	1.78	1.86 (60)	1.86
0.0001	0.01	97.3 (1)	97.3	69.0 (1)	65.9	1.94 (60)	1.94	1.89 (59)	1.89

rule is still near its initial loss except DANP (1.17 on CIFAR-10); at 10^6 passes NP and INP are at 0.107–0.108 on MNIST, within 0.006 of BP's best, and DANP and ANP at 1.09 and 1.16 on CIFAR-10 against 0.907; at 10^7 passes the image-task ordering is that of Table 1. On the LSTM the gap is larger at every budget: after 10^5 passes BP is at 1.24 against 2.17–2.28 for 1.5-ANP, INP and NP, because a backward pass through 64 time steps buys far more than a perturbed forward pass does.

Backpropagation reference. With Adam and no schedule, backpropagation completes 2.7–4.3M steps on MNIST (seed spread) and 2.0M on CIFAR-10 in the hour (Table 19) and overfits the 40k-parameter models within minutes. At $\eta = 10^{-3}$ the MNIST validation loss is lowest at the first evaluation (97.2 % test accuracy, three seeds) and the run ends at 97.4 % with a test loss of 2.4–2.6; the CIFAR-10 run has a validation loss above its initial value from the first evaluation on (its "best checkpoint" is step 0) and is stopped by the divergence rule at 66.9 %. At $\eta = 10^{-4}$ the CIFAR-10 run peaks at 69.2(6) % at minute 1 and ends at 65.6 % with a test loss of 14, and on MNIST $\eta = 10^{-5}$ gives the lowest validation loss (0.102(4) at minute 5, 97.0(2) %, three seeds), which the selection rule therefore picks; the three-seed $\eta = 10^{-4}$ run is equivalent in accuracy (97.2(1) %). The best point of BP on these models thus lies within the first few minutes at any learning rate, and the reference reported in Table 1 is the one the protocol's own rule selects. On the sequence tasks $\eta = 10^{-3}$ is best, BP is still improving at minute 60 (1.80 and 1.84 bits per character) and the reference is unproblematic.

Probe scale and step size. The stage-1 sweep (Appendix E, Figure 8) is decisive about the tied scalar: at $\varepsilon = 0.1$, 15 of the 24 runs of the six original probe methods diverge (all four DANP runs, three each of NP, ANP and RS, two of SPSA, none of INP), BP itself collapses on both image tasks at this step size, and the surviving probe runs make little progress; at $\varepsilon = 0.01$ seven probe runs diverge, five of them node-perturbation runs, and BP is stopped on MNIST; at $\varepsilon = 0.001$ no probe run diverges and 25 of the 32 original pairs select this value. The screening of the two 1.5-order rules under the stage-2 settings follows the same pattern: six of their sixteen runs at $\varepsilon \geq 0.01$ diverge (1.5-SPSA

Table 6: Probe scale decoupled from the step size: test loss of the best validation checkpoint within 30 minutes for every probe method and task. Off-diagonal cells are single 30-minute runs; the tied diagonal cells are one-hour runs at minute 30 (the stage-2 seeds where that value was selected, the stage-1 run otherwise). The $\eta = 10^{-3}$ off-diagonal cells and the ($\eta = 10^{-2}$, $\sigma = 10^{-3}$) cell were run only for the three rules that show them; the last column is the $\sigma = 0.1$ row added at $\eta = 10^{-4}$. A dagger marks a diverged run.

		$\eta = 10^{-2}$			$\eta = 10^{-3}$			$\eta = 10^{-4}$			
Task	Method	σ=10^{-2}	10^{-3}	10^{-4}	σ=10^{-2}	10^{-3}	10^{-4}	σ=10^{-1}	10^{-2}	10^{-3}	10^{-4}
MNIST / MLP	SPSA	2.229	–	2.001	–	0.365	–	0.209	0.165	0.173	**0.154**
	1.5-SPSA	2.226†	2.159	1.977	0.332	0.365	0.392	0.222	0.171	0.180	**0.160**
	RS	2.761	2.496	2.308	–	0.440	–	2.767	0.206	0.208	**0.182**
	NP	0.984	–	0.503	–	0.143	–	**0.106**	0.110	0.112	0.107
	ANP	0.557	–	0.496	–	0.126	–	0.110	0.109	**0.107**	0.112
	1.5-ANP	1.574	1.987	2.101	0.154	0.218	0.324	0.142	**0.127**	0.146	0.167
	INP	0.606†	–	0.370	–	0.134	–	0.111	0.110	0.115	**0.107**
	DANP	0.306	0.299	0.303	–	**0.211**	–	0.227	0.216	0.215	0.215
CIFAR-10 / ConvNet	SPSA	3.234†	–	2.303	–	1.645	–	2.085	1.304	1.353	**1.292**
	1.5-SPSA	2.367†	5.402	2.993	1.525	1.609	1.604	2.106	1.322	1.373	**1.313**
	RS	2.328†	2.406†	9.260†	–	1.730	–	2.406	1.463	1.447	**1.373**
	NP	2.406†	–	2.331†	–	1.407	–	1.227	1.192	1.188	**1.105**
	ANP	2.312	–	2.306	–	1.175	–	1.710	1.262	1.161	**1.122**
	1.5-ANP	2.304	2.304	2.306	**1.360**	1.775	1.860	1.508	1.428	1.473	1.639
	INP	2.304	–	2.303	–	1.315	–	1.597	1.159	1.165	**1.139**
	DANP	1.144	1.170	1.174	–	**1.101**	–	2.081†	1.444	1.258	1.238
PTB / char-LSTM	SPSA	**2.109**	–	2.129	–	2.370	–	3.192	3.100	3.090	3.086
	1.5-SPSA	**2.095**	2.150	2.127	2.217	2.234	2.224	3.231	3.102	3.114	3.107
	RS	2.306	**2.250**	2.264	–	2.483	–	3.508	3.253	3.270	3.272
	NP	2.218	–	**2.205**	–	2.222	–	2.971	2.949	2.946	2.987
	ANP	3.356†	–	2.999	–	**2.961**	–	3.416	3.008	3.007	3.008
	1.5-ANP	2.167	**2.139**	2.139	2.236	2.268	2.235	2.616	2.628	2.718	2.693
	INP	**2.023**	–	2.033	–	2.137	–	2.839	2.837	2.826	2.816
	DANP	3.132†	2.758	2.857	–	**2.730**	–	3.633	3.059	3.051	3.059
PTB / char-Transformer	SPSA	2.536	–	2.580	–	**2.263**	–	2.892	2.348	2.352	2.352
	1.5-SPSA	2.547	2.584	2.570	**2.209**	2.230	2.225	2.920	2.358	2.360	2.359
	RS	2.736	2.792	2.768	–	**2.317**	–	3.551	2.721	2.401	2.400
	NP	2.964	–	2.937	–	**2.372**	–	2.477	2.397	2.399	2.393
	ANP	4.496†	–	2.827	–	3.065	–	3.536†	2.936	**2.719**	2.735
	1.5-ANP	2.703	2.651	2.694	**2.581**	2.593	2.633	2.595	2.675	2.671	2.585
	INP	2.605	–	2.660	–	**2.228**	–	2.345	2.329	2.324	2.332
	DANP	2.588	2.414	**2.384**	–	2.525	–	4.235	3.770	3.066	3.101

on both image tasks at 0.1 and 0.01, 1.5-ANP on CIFAR-10 and the LSTM at 0.1), and under the stage-2 settings every pair of the two 1.5-order rules then selected $\varepsilon = 0.001$ except 1.5-ANP on the LSTM (0.01), before the lower values were added (Table 13). Because ε is both the Adam step and the probe scale, these counts do not say whether a method diverged from noise or from step size, and the published rules do not obey the tie (Dalm et al. [2026] fix $\sigma = 10^{-3}$ with learning rates down to 10^{-5}; MeZO uses $\sigma = 10^{-3}$ with learning rates of 10^{-6}–10^{-7}). Table 6 separates the two, and Table 7 lists the controls under the protocol at $\varepsilon = 10^{-3}$: 1.5-SPSA with a non-inert factor ($\alpha = 0.5$, $\lambda = 10^{-3}$) is within 0.01 of SPSA on the image tasks and 0.02–0.11 worse on the sequence tasks, and 1.5-ANP with $\alpha = 0$ beats 1.5-ANP by 0.08 on MNIST and 0.58 on CIFAR-10 already at this ε, with fp64 losses alone recovering about three quarters of that.

Estimator alignment. Figure 7 measures, on one fixed batch per task, the cosine between each estimator and the true gradient as a function of the number of forward passes spent on it, after a 2000-step backpropagation warm-up, in fp32 on the GPU with TF32 disabled (Appendix F gives the per-layer values and the dependence on the probe scale). On the MLP the node-perturbation rules obtain, from the protocol's 16 passes, a direction with cosine 0.4–0.6 to the gradient, rising to 0.96 (NP), 0.95 (INP), 0.83 (DANP) and 0.81 (ANP) with 1024 passes, whereas the weight-space rules get 0.01 from 16 passes and 0.07–0.11 from 1024, the $\sqrt{K/P}$ scaling of a 40k-dimensional random search. On the ConvNet every estimator is below 0.1 at 16 passes and the node rules reach 0.51–0.53 with 1024: the convolutional layers, whose 26,682 perturbed unit-positions approach the number of weights, need thousands of probes before the node estimate carries signal (Table 22: 0.4–0.6 in the three convolutional layers against 0.8–1.0 in the two dense layers), which is why the

Table 7: Controls under the protocol at $\varepsilon = 10^{-3}$ (Adam, tied scale): test loss (accuracy or bits per character) at the best validation checkpoint of the tied runs of SPSA, 1.5-SPSA and 1.5-ANP (mean over seeds) next to single-seed controls: 1.5-SPSA with a non-inert curvature factor ($\alpha = 0.5$, $\lambda = 10^{-3}$), 1.5-ANP without curvature weighting ($\alpha = 0$) and 1.5-ANP with fp64 losses. The LSTM selection of 1.5-ANP is $\varepsilon = 10^{-2}$, so its 10^{-3} run is listed; its two LSTM controls were not run, and SPSA on CIFAR-10 at 10^{-3} is a single stage-1 run.

Task	SPSA	1.5-SPSA	1.5-SPSA α=0.5, λ=10^{-3}	1.5-ANP	1.5-ANP α=0	1.5-ANP fp64
MNIST / MLP	0.365±0.001	0.365±0.013	0.364 (90.0)	0.224±0.023	0.145 (96.3)	0.187 (95.2)
CIFAR-10 / ConvNet	1.645	1.609±0.011	1.638 (38.7)	1.775±0.018	1.200 (58.1)	1.349 (52.9)
PTB / char-LSTM	2.141±0.082	2.054±0.008	2.249 (3.24)	2.268	–	–
PTB / char-Transformer	2.200±0.017	2.155±0.026	2.216 (3.20)	2.593±0.071	2.536 (3.66)	2.615 (3.77)

Table 8: 1.5-ANP under the protocol's optimizer at the step size the ε screening selected on the image tasks (Adam, $\eta = 10^{-4}$), with the probe scale decoupled from the step size and the losses evaluated in fp64: test loss (accuracy or bits per character) at the best validation checkpoint, seed 0, for $\sigma \in \{10^{-2}, 10^{-3}\}$ and curvature exponent $\alpha \in \{0.5, 0, 0.25, 1\}$ ($\alpha = 0$ removes the curvature weighting); the "damped" row is the LSTM with the recurrent-state damping (0.5) of its specification. The last two columns are the tied-protocol selections of 1.5-ANP and ANP (mean over seeds; the CIFAR-10 selection of 1.5-ANP is $\varepsilon = 10^{-5}$, its 10^{-4} run is within 0.05 of it).

Task	σ=10^{-2}	10^{-2}, α=0	10^{-3}	10^{-3}, α=0	10^{-3}, α=$\frac{1}{4}$	10^{-3}, α=1	1.5-ANP tied	ANP tied
MNIST / MLP	0.132 (96.2)	0.115 (96.8)	0.133 (96.2)	0.112 (97.0)	0.119 (96.5)	0.204 (94.4)	0.159±0.007	0.112±0.003
CIFAR-10 / ConvNet	1.279 (56.9)	1.222 (56.5)	1.307 (54.7)	1.120 (60.4)	1.174 (58.8)	1.981 (31.2)	1.532±0.041	1.058±0.025
PTB / char-LSTM	2.482 (3.58)	2.475 (3.57)	2.537 (3.66)	2.475 (3.57)	2.395 (3.46)	2.848 (4.11)	2.167±0.021	2.885±0.082
PTB / char-LSTM, damped	2.382 (3.44)	–	2.383 (3.44)	–	–	–	–	–
PTB / char-Transformer	2.670 (3.85)	2.552 (3.68)	2.664 (3.84)	2.560 (3.69)	2.587 (3.73)	2.641 (3.81)	2.566±0.085	2.735±0.039

node rules train the ConvNet slowly and the weight-space rules hardly at all. On the LSTM and the Transformer the whole-model cosine stays below 0.26 and 0.17 at 1024 passes (INP reaches 0.40 and 0.33 at 4096): perturbations propagate through shared weights over 64 positions and decorrelate the loss difference from any single unit. The curvature preconditioners behave as their results suggest: 1.5-SPSA is indistinguishable from SPSA at every probe count and every σ, because with $\alpha = 0.1$ and $\lambda = 1$ its per-direction weights span a narrow range whose common scale Adam removes, while 1.5-ANP's per-unit factor ($\alpha = 0.5$) lowers the alignment of ANP from 0.81 to 0.69 on the MLP and from 0.52 to 0.27 on the ConvNet at $\sigma = 10^{-3}$ and raises it on the LSTM (0.26 against 0.04); its second difference is the only quantity that degrades with the probe scale in fp32 (Table 21: 0.74 at $\sigma = 10^{-2}$ to 0.46 at 10^{-5} on the MLP), the first-difference estimators being unchanged down to 10^{-5}. Numerical precision also matters for the convolutions: with TF32, which PyTorch enables by default, the ConvNet cosine of NP at $\sigma = 10^{-3}$ and 1024 passes drops from 0.445 to 0.420 and that of INP from 0.425 to 0.358 (fp64: 0.475 and 0.484), while SPSA and ANP are unaffected and nothing changes at $\sigma = 10^{-2}$ (Table 23, measured on a different batch and warm-up seed from Table 22). Stage 1 ran with TF32 on; stage 2 runs in full fp32.

Sequence models. On the test split a uniform model scores 5.64 bits/char, a unigram 4.32, a bigram 3.31 and an add-0.1 trigram 2.61 (count models estimated on the training text; Appendix G). On the LSTM, INP, 1.5-SPSA, NP, SPSA, 1.5-ANP and RS (2.91–3.22) sit between the bigram and the trigram; FF (3.78) and DANP (3.80) are worse than the bigram and ANP (4.16) is near the unigram. On the Transformer only 1.5-SPSA, INP, SPSA and RS (3.11–3.28) edge past the bigram. BP reaches 1.80 and 1.84 bits per character, well beyond the trigram: after one hour the zero-order rules have learned roughly bigram statistics and backpropagation a real character model. The curves of the leading probe rules are still descending at minute 60, so part of the gap is budget rather than estimator, but at 12–34k steps per hour on the LSTM closing it would take days.

Forward-forward. FF is not a probe method and trains fast (1.7M steps per hour on the MLP), but its two inference rules disagree. The linear readout on the neutral-label features of the 32-unit top block peaks within the first minute on MNIST (87.5 % at $\varepsilon = 0.001$) and then collapses to 14–20 % as its training cross-entropy rises towards $\ln 10$; Hinton's label accumulation on the same trained blocks, summed over all blocks but the first (the protocol's rule, which on this two-block MLP means the second block alone), gives 95.2 % for the screening seed at the selected $\varepsilon = 0.1$ but 59.7(295) % over

Table 9: Untied settings replicated for an hour: for the pairs whose best untied (η, σ) beat the best tied cell of Table 6 by at least 3 %, the one-hour best-checkpoint test loss at that setting (mean over the seeds that completed before the deadline) next to the tied selection of Table 1.

Task	Method	untied (η, σ)	untied (seeds)	tied selection (seeds)	ε^*
MNIST / MLP	ANP	(0.0001, 0.001)	0.110±0.005 (3)	0.112±0.003 (3)	1e-05
MNIST / MLP	1.5-ANP	(0.0001, 0.01)	0.127±0.004 (3)	0.159±0.007 (3)	0.0001
CIFAR-10 / ConvNet	1.5-ANP	(0.001, 0.01)	1.349±0.064 (3)	1.532±0.041 (3)	1e-05
PTB / char-Transformer	DANP	(0.01, 0.0001)	2.389±0.029 (3)	2.578±0.092 (2)	0.01

Table 10: Model-size sweep: best-checkpoint test loss per task and summed, at the ε selected at 40k parameters, one seed (the 40k rows are the seed means of Table 1). Widths: MLP hidden (400, 240) and (1600, 1700); ConvNet channels (48, 96, 96) and (128, 256, 256); LSTM hidden 265 and 870; Transformer width 152 and 496.

Params	Task	BP	SPSA	1.5-SPSA	RS	NP	ANP	1.5-ANP	INP	DANP	FF
40k	MNIST / MLP	0.102	0.142	0.140	0.168	0.107	0.112	0.159	0.107	0.211	2.335
	CIFAR-10 / ConvNet	0.907	1.200	1.215	1.292	1.011	1.058	1.532	1.055	1.101	2.007
	PTB / char-LSTM	1.245	2.141	2.054	2.229	2.092	2.885	2.167	2.017	2.635	2.618
	PTB / char-Transformer	1.275	2.200	2.155	2.276	2.328	2.735	2.566	2.176	2.578	2.932
	sum	3.529	5.683	5.563	5.965	5.539	6.791	6.425	5.355	6.526	9.893
400k	MNIST / MLP	0.071	0.141	0.141	0.180	0.071	0.073	0.237	0.095	0.182	2.302
	CIFAR-10 / ConvNet	2.224	1.237	1.265	1.387	1.037	1.005	1.468	1.062	1.028	1.821
	PTB / char-LSTM	1.047	2.059	2.059	2.140	2.086	2.898	2.088	1.886	2.567	2.525
	PTB / char-Transformer	1.058	2.297	2.329	2.391	2.273	2.646	2.634	2.357	2.772	2.856
	sum	4.400	5.734	5.794	6.097	5.466	6.621	6.426	5.400	6.549	9.504
4M	MNIST / MLP	0.132	0.187	0.190	0.226	0.067	0.072	0.534	0.147	0.232	2.139
	CIFAR-10 / ConvNet	1.920	1.409	1.417	1.510	1.164	1.011	1.555	1.200	1.042	1.771
	PTB / char-LSTM	1.021	2.128	2.159	2.256	2.214	3.184	2.113	2.111	2.402	2.626
	PTB / char-Transformer	1.067	2.479	2.476	2.586	2.541	2.614	2.655	2.688	2.934	2.795
	sum	4.140	6.203	6.241	6.577	5.985	6.881	6.858	6.146	6.610	9.330

three seeds (the other two average 42 %) and 93.1(3) % at $\varepsilon = 0.01$ (Table 13). On CIFAR-10 both numbers are 27–34 %, and on the sequence tasks FF with corrupted-sequence negatives is third-worst on the LSTM and worst on the Transformer (3.78 and 4.23 bits per character): its goodness separates real from corrupted text, but the normalised features support only a weak next-character readout.

Faithfulness. SPSA and 1.5-SPSA use Gaussian rather than Rademacher directions and RS is expressed through Adam (Table 11: the direct-jump variant); 1.5-ANP applies the recurrent-state damping of its specification only in the controls of Table 8; FF's goodness, threshold and readout follow the reference re-implementation and the parameter budget rather than Hinton's 2,000-unit layers; DANP's decorrelation rate is fixed and un-normalised, which is why it reaches NaN within 21 steps at $\varepsilon = 0.1$ on both image models (a single Adam step inflates the ReLU activities by an order of magnitude and the $\mathbf{R}_l$ update then grows tenfold per step). All of these are stated where the method is defined.

D Extended related work

This appendix is the unabridged version of Section 2.

Weight-space perturbation. Simultaneous perturbation stochastic approximation [Spall, 1992, 1998] estimates a gradient from two loss evaluations at $\boldsymbol{\theta} \pm \sigma \mathbf{z}$ with a random direction $\mathbf{z}$; with Gaussian $\mathbf{z}$ the estimator is the gradient of the Gaussian-smoothed loss [Nesterov and Spokoiny, 2017], with complexity understood in the convex [Duchi et al., 2015] and bandit [Flaxman et al., 2005] settings (see Liu et al. [2020] for a survey). Weight perturbation was proposed for analog VLSI training by Jabri and Flower [1992], with stochastic error descent [Cauwenberghs, 1992] and summed-weight perturbation [Flower and Jabri, 1992] as early variance reductions. Evolution strategies use the same estimator with many directions, fitness shaping and natural-gradient corrections [Wierstra

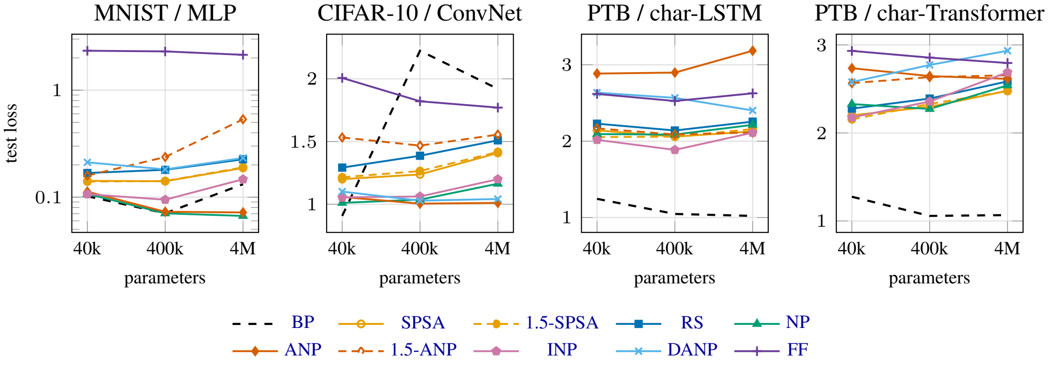

Figure 6: Per-task best-checkpoint test loss against model size (one seed at 400k and 4M, seed mean at 40k; ε selected at 40k).

Table 11: Follow-up runs (seed 0, 1 h). Top: random search through Adam (protocol) against the direct jump $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \sigma \mathbf{z}_{k^*}$ at the three model sizes. Cells marked "–" were still running at the time of this build.

Params	Task	RS (Adam, protocol)	RS direct jump
40k	MNIST / MLP	0.168±0.012 (94.9±0.3)	–
	CIFAR-10 / ConvNet	1.292±0.056 (53.7±2.2)	–
	PTB / char-LSTM	2.229±0.092 (3.22±0.13)	–
	PTB / char-Transformer	2.276±0.027 (3.28±0.04)	–
400k	MNIST / MLP	0.180 (94.4)	–
	CIFAR-10 / ConvNet	1.387 (50.9)	–
	PTB / char-LSTM	2.140 (3.09)	–
	PTB / char-Transformer	2.391 (3.45)	–
4M	MNIST / MLP	0.226 (93.3)	–
	CIFAR-10 / ConvNet	1.510 (46.7)	–
	PTB / char-LSTM	2.256 (3.25)	–
	PTB / char-Transformer	2.586 (3.73)	–

et al., 2014, Hansen et al., 2003, Hansen, 2016]; Salimans et al. [2017] and Mania et al. [2018] showed them competitive with policy-gradient reinforcement learning, Zhang et al. [2017] related them to SGD, and Lenc et al. [2019] applied them to supervised learning. Adaptive-moment updates on top of zero-order estimates were studied by Chen et al. [2019], and Silver et al. [2022] analysed learning from directional derivatives along random directions. MeZO [Malladi et al., 2023] fine-tunes billion-parameter language models with two-point SPSA and in-place perturbation; 1.5-SPSA [Chaubard, 2025b] adds a three-point curvature estimate along each direction to damp the step in sharp regions. DeepZero [Chen et al., 2024] and Zhang et al. [2024] benchmark zero-order estimators on large pretrained models; we vary the estimator *and* the architecture at a scale where every method can be run to its one-hour limit. Our SPSA, 1.5-SPSA and RS baselines are the two-point, curvature-preconditioned and best-of-K instances of this family.

Node perturbation. Node perturbation injects noise into the units of the network and correlates the resulting loss change with the noise, which yields a weight update local to each layer [Widrow and Lehr, 1990, Werfel et al., 2003, 2005, Fiete and Seung, 2006]. Hiratani et al. [2022] analysed its stability and showed that it suffers from weight diffusion in mismatched networks; Züge et al. [2023] compared weight and node perturbation on temporally extended tasks and found weight perturbation often similar or better, a finding our sequence tasks revisit. Dalm et al. [2026] reformulated node perturbation as a directional derivative (*iterative* NP), proposed *activity-based* NP that uses the measured activity change rather than the injected noise, and combined it with the input decorrelation rule of Ahmad et al. [2023], which also accelerates backpropagation [Dalm et al., 2024, Ahmad, 2024]. 1.5-ANP [Chaubard, 2025a] applies a three-point curvature preconditioner to central-difference activity perturbation. We implement all five variants.

Table 12: Follow-up runs (seed 0, 1 h): 65 forward passes per step at Adam $\eta = 10^{-4}$, $\sigma = 10^{-3}$ (1.5-ANP with 32 antithetic pairs and fp64 losses, with its $\alpha = 0$ control; ANP and INP with 64 noisy passes) against the 17- and 16-pass tied-protocol selections of Table 1. Cells marked "–" were still running at the time of this build.

Task	1.5-ANP 65	1.5-ANP 65, α=0	ANP 65	INP 65	1.5-ANP 17	ANP 16	INP 16
MNIST / MLP	–	–	–	–	0.159±0.007	0.112±0.003	0.107±0.004
CIFAR-10 / ConvNet	–	–	–	–	1.532±0.041	1.058±0.025	1.055±0.015
PTB / char-LSTM	–	–	–	–	2.167±0.021	2.885±0.082	2.017±0.055
PTB / char-Transformer	–	–	–	–	2.566±0.085	2.735±0.039	2.176±0.022

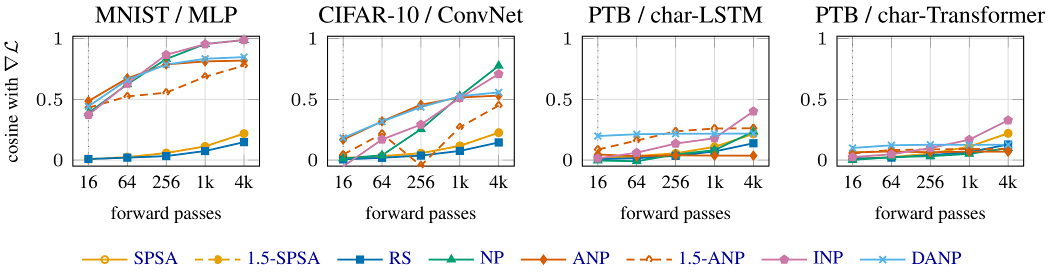

Figure 7: Cosine between each estimator and the true gradient of one batch (whole parameter vector) versus the number of forward passes spent, $\sigma = 10^{-3}$, after a 2000-step backpropagation warm-up. The protocol spends 16–17 passes per step.

Local rules and benchmarks. The forward-forward algorithm [Hinton, 2022] replaces the backward pass by two forward passes with positive and negative data and trains each layer to separate them with a local "goodness" objective. Rules that broadcast a global error vector [Clark et al., 2021], feedback alignment [Lillicrap et al., 2016, Nøkland, 2016] and synthetic gradients [Jaderberg et al., 2017] still require a learned or random backward signal, and forward gradients [Baydin et al., 2022, Ren et al., 2023] require a Jacobian-vector product through the network; they are outside the zero-order scope of this benchmark, which includes forward-forward as the representative local rule. Controlled comparisons of biologically motivated rules exist for the feedback-alignment family [Bartunov et al., 2018, Launay et al., 2020, Sanfiz and Akrout, 2021]; we are not aware of one that puts weight perturbation, the node-perturbation family and forward-forward on the same models under one budget.

E Stage-1 screening grid

Table 14 lists the final validation and test loss of every stage-1 run of the eight original methods, Figure 8 the final test loss against ε, and Figure 9 every run's test-loss curve. The validation and test values of a collapsed run can differ by orders of magnitude at the same evaluation because the loss of a collapsed network is dominated by a few examples with huge logits. The two 1.5-order rules and the values $\varepsilon \leq 10^{-4}$ of every method were screened under the stage-2 settings (1-minute evaluations, fp32, 1 h); Table 13 gives the validation loss of every value and the resulting selections.

F Selected configurations, throughput and ablation tables

Table 15 lists the selected ε and the minute of the best checkpoint of every pair, Table 17 the best-validation-*accuracy* checkpoints of the two classification tasks, Table 16 the end-of-hour values, Tables 18–20 the throughput, step counts and peak memory, and Tables 21–23 the alignment against the probe scale, the per-layer alignment and the precision check.

G Implementation details and hyper-parameters

Perturbable layers. Every trainable layer of every model is a perturbable linear, convolutional or embedding layer. A global context selects what a layer does on its forward pass: (i) a plain

Table 13: Tied ε screening of every (task, method) pair: the lowest validation loss over the hour at each value (FF on the image tasks: goodness validation accuracy), with the selected value in bold. Values at $\varepsilon \leq 10^{-4}$ come from one-hour runs under the stage-2 settings; the first three columns are stage-1 runs where no stage-2 run exists. Every selected value has a worse neighbour on each side except at a grid edge (10^{-5} for four image-task pairs, 0.1 for FF on MNIST).

Task	Method	$\varepsilon = 0.1$	$\varepsilon = 0.01$	$\varepsilon = 0.001$	$\varepsilon = 0.0001$	$\varepsilon = 10^{-5}$
MNIST / MLP	BP	1.826	2.744	0.314	0.100	**0.094**
	SPSA	2.566	2.087	0.285	**0.132**	0.185
	1.5-SPSA	2.303	2.048	0.287	**0.134**	0.196
	RS	2.744	2.156	0.360	**0.140**	0.206
	NP	2.301	0.752	0.142	0.102	**0.099**
	ANP	2.304	0.446	0.127	0.102	**0.100**
	1.5-ANP	2.304	1.454	0.176	**0.133**	0.140
	INP	2.305	0.396	0.128	**0.098**	0.101
	DANP	2.744	0.242	**0.181**	0.183	–
	FF	**0.952**	0.948	0.918	0.922	–
CIFAR-10 / ConvNet	BP	2.305	1.183	2.395	**0.867**	0.881
	SPSA	2.395	2.303	1.622	**1.268**	1.529
	1.5-SPSA	2.395	2.303	1.586	**1.281**	1.563
	RS	2.395	2.327	1.738	**1.346**	1.591
	NP	2.395	2.395	1.400	**1.059**	1.222
	ANP	2.395	2.303	1.064	**1.022**	1.224
	1.5-ANP	2.395	2.303	1.775	1.538	**1.485**
	INP	2.305	2.302	1.265	**1.052**	1.183
	DANP	2.395	1.129	**1.052**	1.158	–
	FF	0.224	**0.338**	0.334	0.135	–
PTB / char-LSTM	BP	1.765	1.315	**1.268**	1.492	–
	SPSA	3.597	2.117	**2.066**	2.993	–
	1.5-SPSA	3.702	2.102	**2.067**	2.998	–
	RS	3.911	2.314	**2.141**	3.033	–
	NP	3.911	2.239	**2.033**	2.745	–
	ANP	3.911	3.382	**2.970**	2.999	–
	1.5-ANP	2.494	**2.165**	2.287	2.518	–
	INP	2.652	2.007	**1.971**	2.563	–
	DANP	3.911	3.151	**2.627**	2.858	–
	FF	2.832	**2.674**	2.789	2.956	–
PTB / char-Transformer	BP	1.731	1.344	**1.299**	1.371	–
	SPSA	4.497	2.554	**2.201**	2.296	–
	1.5-SPSA	4.497	2.561	**2.191**	2.301	–
	RS	4.497	2.728	**2.297**	2.347	–
	NP	4.497	2.957	2.359	**2.353**	2.732
	ANP	4.497	4.497	3.060	**2.721**	2.851
	1.5-ANP	3.330	2.704	2.621	**2.602**	2.701
	INP	4.497	2.613	**2.170**	2.284	–
	DANP	4.497	**2.530**	2.530	2.996	–
	FF	3.046	2.924	2.946	**2.919**	2.946

differentiable forward (backpropagation, forward-forward local losses, evaluation); (ii) a *clean* forward that detaches the layer input and records the input $\mathbf{h}_{l-1}$ and pre-activation $\mathbf{a}_l$, keeping a local autograd graph to the layer's own parameters only; or (iii) a graph-free *noisy* forward that adds $\pm\sigma\boldsymbol{\xi}_l$ to the pre-activation of every layer (or of one selected layer for INP; the minus sign reuses the probes of the plus pass for the antithetic pairs of 1.5-ANP) and records $\boldsymbol{\xi}_l$ and $\tilde{\mathbf{a}}_l$. Given a per-unit credit tensor $\mathbf{G}_l$, the weight update $\mathbf{G}_l^\top \mathbf{h}_{l-1}$ and its convolutional and embedding analogues are obtained by a one-layer backward through the recorded local graph, so no error is propagated through any nonlinearity. A layer called several times in one forward (the LSTM gate layer, once per time step) accumulates one record per call.

Table 14: Stage 1: final validation / test loss of all 96 runs of the original eight methods (seed 0, 5-minute evaluations). Bold: the ε with the lowest *final* validation loss. A dagger marks a diverged run (last finite evaluation shown). Initial validation losses: 2.744 (MNIST), 2.395 (CIFAR-10), 3.911 (LSTM), 4.497 (Transformer).

		validation / test loss		
Task	Optimizer	$\varepsilon = 0.1$	$\varepsilon = 0.01$	$\varepsilon = 0.001$
MNIST / MLP	BP	**1.826 / 1.857**	51.107 / 177.402†	2.239 / 2.355
	SPSA	957.457 / 2.302†	2.193 / 5.502	**0.917 / 1.039**
	RS	52.924 / 170.573†	8.157 / 4.833	**1.238 / 1.412**
	NP	2.301 / 2.301†	2.303 / 2.302	**1.985 / 2.141**
	ANP	2.323 / 2.319	0.519 / 0.619	**0.363 / 0.381**
	INP	2.325 / 2.321	8.917 / 21.010†	**0.935 / 0.916**
	DANP	– / –†	1.033 / 1.190	**0.643 / 0.751**
	FF	2.288 / 2.288	2.243 / 2.241	**2.123 / 2.127**
CIFAR-10 / ConvNet	BP	2.305 / 2.305	**1.357 / 2.402**	23.735 / 24.475†
	SPSA	7.292 / 7.225†	22.978 / 2.313†	**2.300 / 2.468**
	RS	**2.395 / 2.406**	197.361 / 38.724†	2.401 / 2.492
	NP	1.9×10^6 / $1.3\times10^{6\dagger}$	314.593 / 326.902†	**1.597 / 1.613**
	ANP	23.474 / 23.143†	2.332 / 2.336	**1.143 / 1.177**
	INP	2.327 / 2.323	2.303 / 2.303	**1.321 / 1.339**
	DANP	– / –†	**1.242 / 1.285**	1.289 / 1.371
	FF	2.311 / 2.311	2.081 / 2.084	**1.880 / 1.886**
PTB / char-LSTM	BP	1.858 / 1.835	1.371 / 1.340	**1.268 / 1.231**
	SPSA	3.991 / 3.965	2.125 / 2.107	**2.066 / 2.048**
	RS	15.802 / 15.689†	2.330 / 2.310	**2.141 / 2.124**
	NP	23.036 / 22.815†	2.380 / 2.356	**2.033 / 2.015**
	ANP	195.280 / 192.326†	9.592 / 9.603†	**3.242 / 3.232**
	INP	2.885 / 2.866	2.007 / 1.989	**1.971 / 1.955**
	DANP	271.941 / 271.567†	13.384 / 13.251†	**2.627 / 2.608**
	FF	2.862 / 2.859	**2.699 / 2.688**	2.789 / 2.776
PTB / char-Transformer	BP	1.802 / 1.780	1.351 / 1.316	**1.300 / 1.262**
	SPSA	6.865 / 6.893	2.557 / 2.539	**2.201 / 2.181**
	RS	9.453 / 9.538†	2.745 / 2.726	**2.297 / 2.279**
	NP	6.617 / 6.746	3.206 / 3.180	**2.374 / 2.357**
	ANP	305.304 / 306.216†	27.783 / 27.601†	**3.433 / 3.412**
	INP	5.921 / 5.880	2.651 / 2.638	**2.170 / 2.151**
	DANP	26.685 / 26.551†	**2.543 / 2.525**	2.564 / 2.554
	FF	5.707 / 5.694	**2.958 / 2.959**	2.976 / 2.983

Decorrelation. Each linear and convolutional layer of DANP owns a square matrix $\mathbf{R}_l$ (identity-initialised buffer, not an optimizer parameter) that multiplies the layer input; for convolutions the decorrelated vector is the unfolded $C_{\text{in}}k^2$ patch and the decorrelation is itself a convolution. $\mathbf{R}_l$ is updated once per step with Eq. (9) from the inputs recorded on the clean pass, from the first step. Embeddings are not decorrelated.

1.5-ANP. Attribution units are examples (images) or (sequence, position) tokens (text). The per-token loss is the next-character cross-entropy at that position; the per-token norm $\|\mathbf{d}_u\|^2$ sums the squared activity differences of every layer at that position (token embedding; the gate pre-activations of the LSTM call at that time step; the projections and feed-forward layers of the Transformer at that position; the readout), and the position embedding, perturbed once per batch, contributes its row for that position to every sequence and receives the sum over sequences of β_u. The noisy passes run the perturbed recurrent state without the specification's 0.5 damping. $\lambda = 10^{-3}$, $\alpha = 0.5$, $\epsilon_{\text{mach}} = 10^{-12}$.

Language-model specifics. The LSTM is written with a single perturbable layer for the four gate pre-activations so that the gates are the perturbable nodes. The Transformer uses pre-LN blocks with

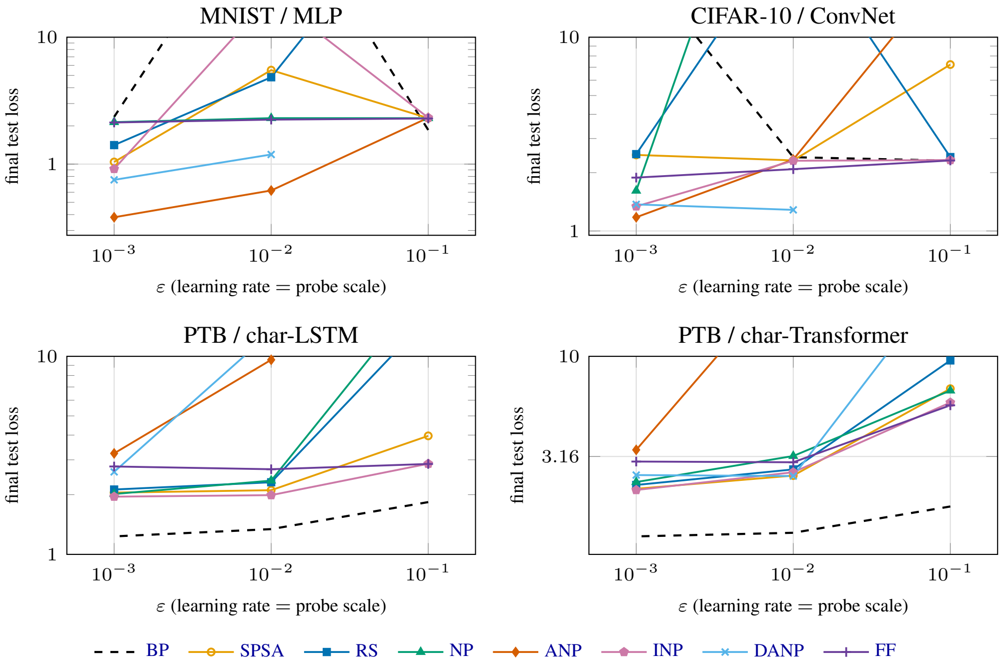

Figure 8: Stage 1: final test loss versus ε (learning rate = probe scale) for the original eight methods; diverged runs are plotted at their last finite evaluation and the y-axis is capped at 10 (values up to 10^6 leave the frame).

Table 15: Stage 2: selected ε^*, mean minute of the best validation checkpoint and number of seeds for every (task, method) pair.

	MNIST		CIFAR-10		LSTM		Transformer	
Method	ε^*	min / seeds	ε^*	min / seeds	ε^*	min / seeds	ε^*	min / seeds
BP	1e-05	5 / 3	0.0001	1 / 3	0.001	60 / 3	0.001	57 / 3
SPSA	0.0001	52 / 3	0.0001	60 / 3	0.001	60 / 3	0.001	60 / 3
1.5-SPSA	0.0001	58 / 3	0.0001	59 / 3	0.001	60 / 3	0.001	59 / 3
RS	0.0001	46 / 3	0.0001	59 / 3	0.001	60 / 3	0.001	60 / 3
NP	1e-05	52 / 3	0.0001	59 / 3	0.001	60 / 3	0.0001	60 / 3
ANP	1e-05	56 / 3	0.0001	58 / 3	0.001	40 / 3	0.0001	21 / 3
1.5-ANP	0.0001	29 / 3	1e-05	60 / 3	0.01	10 / 3	0.0001	44 / 3
INP	0.0001	7 / 3	0.0001	56 / 3	0.001	60 / 3	0.001	60 / 3
DANP	0.001	1 / 3	0.001	5 / 3	0.001	44 / 3	0.01	40 / 2
FF	0.1	29 / 3	0.01	19 / 3	0.01	18 / 3	0.0001	57 / 2

parameter-free layer normalisation, a learned positional embedding over $T = 64$ positions, and a final parameter-free layer normalisation before the readout. The n-gram reference points of Section 5.7 are count models estimated on the training characters and evaluated on the test characters: uniform over $V = 50$, maximum-likelihood unigram and bigram, and a trigram with add-0.1 smoothing.

Forward-forward specifics. One Adam instance updates all parameters; every block's goodness loss is computed on the detached, normalised output of the previous block, so each parameter receives exactly one local gradient per step (the embeddings are trained by the first block's goodness). The label overlay follows Pezeshki [2023]: the first 10 entries of the flattened input are zeroed and the entry of the label is set to the maximum input value (for CIFAR-10, the first 10 pixels of the first row of channel 0); a wrong label is drawn uniformly from the nine other classes; the neutral label sets all 10 entries to 0.1 times the maximum. For PTB, 30 % of the positions of each sequence are replaced by uniformly random characters, resampled every step. Label accumulation sums the goodness over

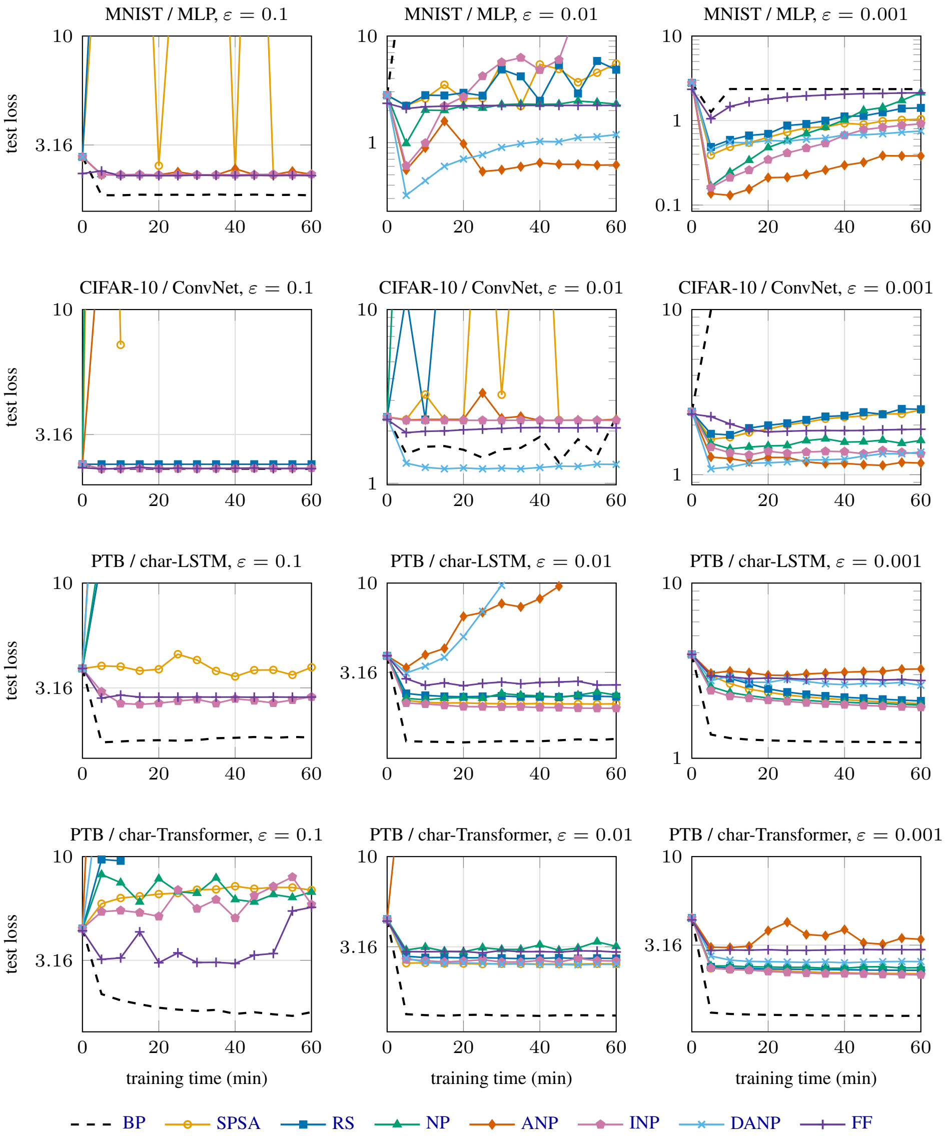

Figure 9: Stage 1: test loss versus training time for every run of the original eight methods. Rows: tasks; columns: $\varepsilon \in \{0.1, 0.01, 0.001\}$.

all blocks but the first (Hinton) or over all blocks (logged as a second number). The readout is trained every step by cross-entropy on the normalised, detached top-block activity.

Evaluation and checkpoints. Validation and test metrics are computed on the full splits at initialisation, every minute of training time (every 5 minutes in stage 1) and at the end; the clock is paused during evaluation. Stage 2 saves the parameters at the best validation loss, at 5 minutes and at the end (released with the logs). Runs are not bitwise reproducible from the seed because cuDNN autotuning is on and stopping is time-based.

Hyper-parameters. Table 24 summarises every hyper-parameter; none is tuned per method beyond the ε screening.

Data and licences. MNIST [LeCun et al., 1998], CIFAR-10 [Krizhevsky, 2009] and the character-level Penn Treebank files [Marcus et al., 1993, Mikolov, 2012] are used as distributed by their authors. MNIST is distributed under the Creative Commons Attribution-Share Alike 3.0 licence; CIFAR-10 is

Table 16: The same runs at the end of the hour (secondary, stability metric): test loss and accuracy or bits per character after 60 minutes of training. A dagger marks a pair in which at least one seed was stopped by the divergence rule (its last evaluation is used).

	MNIST / MLP		CIFAR-10 / ConvNet		PTB / char-LSTM		PTB / char-Transformer		
Method	loss	acc. (%)	loss	acc. (%)	loss	bits/char	loss	bits/char	score
NP	**0.106±0.002**	**96.8±0.1**	**1.035±0.058**	**64.2±2.5**	2.092±0.067	3.02±0.10	2.328±0.011	3.36±0.02	5.56
1.5-SPSA	0.141±0.006	95.9±0.2	1.216±0.047	56.6±1.8	2.054±0.008	2.96±0.01	**2.154±0.025**	**3.11±0.04**	5.57
INP	0.317±0.065	96.9±0.2	1.065±0.017	62.9±0.8	**2.019±0.056**	**2.91±0.08**	2.176±0.022	3.14±0.03	5.58
SPSA	0.139±0.005	96.0±0.2	1.200±0.050	57.7±1.8	2.141±0.082	3.09±0.12	2.200±0.017	3.17±0.02	5.68
RS	0.161±0.009	95.2±0.2	1.295±0.055	53.5±2.3	2.229±0.092	3.22±0.13	2.276±0.027	3.28±0.04	5.96
1.5-ANP	0.163±0.005	96.2±0.4	1.532±0.041	46.2±1.0	2.368±0.068	3.42±0.10	2.687±0.176	3.88±0.25	6.75
ANP	0.111±0.005	96.6±0.2	1.068±0.012	63.1±0.6	3.116±0.273	4.50±0.39	2.909±0.124	4.20±0.18	7.20
DANP	0.782±0.024	94.2±0.2	1.355±0.026	64.2±0.8	2.674±0.058	3.86±0.08	2.735±0.297	3.95±0.43	7.55
FF	2.298±0.010	57.7±29.9 [10.9]	2.091±0.022	32.1±0.6 [23.0]	2.777±0.084	4.01±0.12	2.943±0.029	4.25±0.04	–
BP	0.819±0.209	96.7±0.2	14.311±2.756	65.6±0.3	1.245±0.012	1.80±0.02	1.276±0.012	1.84±0.02	17.65

Table 17: Image tasks: test accuracy (%) and test loss at the evaluation with the highest validation *accuracy*, mean over seeds. Best probe method per task in bold.

	MNIST / MLP		CIFAR-10 / ConvNet	
Method	acc. (%)	loss	acc. (%)	loss
BP	97.1±0.2	0.103±0.003	70.0±0.4	0.964±0.053
SPSA	95.8±0.2	0.143±0.009	57.7±1.8	1.203±0.050
1.5-SPSA	95.7±0.2	0.145±0.009	56.4±1.6	1.222±0.044
RS	95.0±0.3	0.164±0.010	53.6±2.1	1.298±0.052
NP	96.8±0.1	0.107±0.003	**65.2±2.7**	1.006±0.065
ANP	96.6±0.2	0.112±0.004	63.4±1.2	1.055±0.029
1.5-ANP	96.0±0.4	0.161±0.007	45.7±0.9	1.540±0.034
INP	**97.0±0.1**	0.242±0.061	62.9±0.9	1.068±0.030
DANP	94.0±0.0	0.490±0.246	63.2±0.8	1.292±0.024
FF	59.7±29.5	2.335±0.058	33.9±0.3	2.007±0.036

released by its authors for research use without a formal licence; the character-level PTB files are the widely redistributed Mikolov preprocessing of the Penn Treebank (LDC99T42) and are used for research only. Our code is released under the MIT licence.

Compute. Stage 1 used about 100 GPU-hours (10.5 hours of wall-clock on the 8-GPU node); stage 2 and the ablations about 500 GPU-hours (24 hours of wall-clock on up to five 8-GPU nodes); the alignment diagnostic takes about an hour on one GPU. Stage 1 was preceded by 32 two-minute smoke runs and four 10-minute backpropagation runs with 30-second evaluations (released with the logs, superseded by Table 5).

Table 18: Throughput: optimizer steps and forward passes per hour (thousands), mean over the seeds of the selected runs, measured with eight runs sharing one host (one BP step is counted as one pass; its backward is not counted).

	MNIST		CIFAR-10		LSTM		Transformer	
Method	steps/h	passes/h	steps/h	passes/h	steps/h	passes/h	steps/h	passes/h
BP	2683.4	2683.4	2023.8	2023.8	91.6	91.6	560.5	560.5
SPSA	1002.7	16043.3	590.4	9446.4	24.6	394.2	165.1	2641.7
1.5-SPSA	850.3	14455.1	511.2	8690.2	33.6	571.8	227.2	3862.4
RS	1075.8	17212.5	609.0	9743.7	24.4	390.6	170.8	2732.5
NP	503.9	8062.5	336.9	5390.0	17.2	274.7	78.3	1252.3
ANP	365.4	5846.5	264.6	4234.1	14.4	229.7	63.5	1016.7
1.5-ANP	502.9	8549.5	255.8	4348.6	23.5	399.0	73.5	1248.8
INP	598.7	9579.0	277.0	4431.8	18.3	292.9	130.2	2083.3
DANP	430.3	6885.3	250.4	4007.0	12.4	198.8	71.7	1146.8
FF	1761.5	4657.4	958.1	2874.4	81.1	162.3	267.2	534.4

Table 19: Steps completed in the hour (thousands) and milliseconds of training time per step, mean over seeds of the selected runs (eight runs sharing one host).

	MNIST		CIFAR-10		LSTM		Transformer	
Method	steps (k)	ms/step	steps (k)	ms/step	steps (k)	ms/step	steps (k)	ms/step
BP	2683(1383)	1.55	2024(761)	2.02	92(49)	46.19	560(399)	8.44
SPSA	1003(312)	3.89	590(173)	6.55	25(8)	156.07	165(47)	22.86
1.5-SPSA	850(264)	4.59	511(149)	7.56	34(1)	107.12	227(2)	15.85
RS	1076(357)	3.68	609(177)	6.34	24(9)	158.92	171(51)	22.19
NP	504(170)	7.63	337(88)	11.28	17(6)	222.80	78(3)	46.05
ANP	365(111)	10.39	265(76)	14.55	14(5)	267.07	64(2)	56.70
1.5-ANP	503(161)	7.81	256(84)	14.98	23(0)	153.44	73(2)	49.04
INP	599(188)	6.54	277(6)	13.00	18(6)	208.25	130(33)	28.71
DANP	430(7)	8.37	250(7)	14.38	12(4)	307.48	72(21)	52.55
FF	1762(106)	2.05	958(18)	3.76	81(51)	54.87	267(5)	13.48

Table 20: Peak GPU memory of the training process (MB): tensors allocated, and reserved by the caching allocator, recorded during the stage-2 runs.

	MNIST		CIFAR-10		LSTM		Transformer	
Method	alloc.	reserved	alloc.	reserved	alloc.	reserved	alloc.	reserved
BP	243	258	1889	1972	127	142	329	436
SPSA	235	258	1889	1972	118	142	321	420
1.5-SPSA	235	258	1889	1972	118	142	321	420
RS	235	258	1889	1972	118	142	321	420
NP	243	258	1889	1972	151	172	369	482
ANP	243	258	1889	1972	151	172	369	490
1.5-ANP	243	258	1889	1972	148	178	369	538
INP	243	258	1889	1972	145	166	359	466
DANP	247	278	3625	3894	154	192	373	494
FF	249	278	1937	2006	135	178	353	430

Table 21: Whole-model cosine between each estimator and the true gradient at 1024 forward passes (trained state) as a function of the probe scale, fp32 on the GPU. Only 1.5-ANP, whose weights use a second difference, degrades as σ shrinks.

Model	Estimator	$\sigma = 0.1$	$\sigma = 0.01$	$\sigma = 0.001$	$\sigma = 0.0001$	$\sigma = 10^{-5}$
MNIST / MLP	SPSA	0.02	0.11	0.11	0.11	0.11
	RS	0.00	0.07	0.07	0.07	0.07
	NP	0.96	0.96	0.96	0.96	0.96
	ANP	0.81	0.81	0.81	0.81	0.81
	INP	0.95	0.95	0.95	0.95	0.95
	DANP	0.82	0.83	0.83	0.83	0.83
	1.5-SPSA	0.02	0.11	0.11	0.11	0.11
	1.5-ANP	0.78	0.74	0.69	0.54	0.46
CIFAR-10 / ConvNet	SPSA	0.02	0.12	0.12	0.12	0.12
	RS	0.00	0.06	0.08	0.08	0.08
	NP	0.39	0.50	0.53	0.52	0.53
	ANP	0.12	0.22	0.52	0.52	0.52
	INP	0.42	0.54	0.51	0.52	0.51
	DANP	-0.01	0.29	0.53	0.52	0.52
	1.5-SPSA	0.02	0.12	0.12	0.12	0.11
	1.5-ANP	0.49	0.41	0.27	0.25	0.26
PTB / char-LSTM	SPSA	0.02	0.11	0.11	0.11	0.11
	RS	0.00	0.02	0.07	0.07	0.07
	NP	0.07	0.08	0.08	0.08	0.08
	ANP	0.02	0.04	0.04	0.04	0.04
	INP	0.17	0.18	0.18	0.18	0.18
	DANP	0.13	0.22	0.22	0.22	0.22
	1.5-SPSA	0.02	0.11	0.11	0.11	0.10
	1.5-ANP	0.28	0.29	0.26	0.11	0.12
PTB / char-Transformer	SPSA	0.01	0.09	0.11	0.11	0.11
	RS	0.00	0.00	0.07	0.07	0.07
	NP	-0.00	0.03	0.05	0.05	0.05
	ANP	0.01	0.02	0.07	0.07	0.07
	INP	0.02	0.06	0.17	0.18	0.17
	DANP	-0.03	0.08	0.13	0.13	0.13
	1.5-SPSA	0.01	0.09	0.11	0.11	0.11
	1.5-ANP	0.03	0.09	0.09	0.08	0.05

Table 22: Per-layer cosine between each estimator and the true gradient at 1024 forward passes, $\sigma = 10^{-3}$, after the 2000-step backpropagation warm-up (layer indices in module order; “all” is the whole parameter vector).

Model	Layer	SPSA	RS	NP	ANP	INP	DANP	1.5-SPSA	1.5-ANP
MNIST / MLP	0	0.08	0.05	0.95	0.94	0.93	0.94	0.08	0.48
	1	0.30	0.23	0.95	0.90	0.96	0.89	0.30	0.78
	2	0.56	0.35	0.98	0.89	0.99	0.93	0.56	0.83
	all	0.11	0.07	0.96	0.81	0.95	0.83	0.11	0.69
CIFAR-10 / ConvNet	0	0.65	0.41	0.56	0.55	0.46	0.44	0.65	-0.10
	1	0.15	0.10	0.43	0.54	0.48	0.54	0.15	0.10
	2	0.10	0.05	0.45	0.62	0.60	0.60	0.10	0.24
	3	0.04	0.04	0.81	0.93	0.93	0.86	0.04	0.87
	4	0.21	0.15	0.94	0.88	0.99	0.85	0.21	0.87
	all	0.12	0.08	0.53	0.52	0.51	0.53	0.12	0.27
PTB / char-LSTM	0	0.12	0.11	0.58	0.58	0.41	0.97	0.12	0.42
	1	0.08	0.05	0.07	0.04	0.16	0.17	0.08	0.21
	2	0.23	0.15	0.13	0.05	0.27	0.49	0.23	0.36
	all	0.11	0.07	0.08	0.04	0.18	0.22	0.11	0.26
PTB / char-Transformer	0	0.21	0.09	0.44	0.45	0.22	0.81	0.21	0.45
	1	0.15	0.10	0.34	0.33	0.12	0.42	0.15	0.20
	2	0.06	-0.02	0.07	0.15	0.12	0.10	0.06	0.18
	3	0.13	0.10	0.04	0.50	0.11	0.35	0.13	0.38
	4	0.11	0.06	0.03	0.20	0.23	0.45	0.11	0.26
	5	0.17	0.12	0.13	0.16	0.18	0.40	0.17	0.15
	6	0.10	0.11	0.00	0.12	0.21	0.39	0.11	0.14
	7	0.10	0.07	-0.01	0.23	0.19	0.54	0.10	0.23
	8	0.03	0.01	0.05	0.20	0.32	0.06	0.03	0.15
	9	0.08	0.02	0.07	0.03	0.16	0.07	0.08	-0.03
	10	0.11	0.08	0.10	0.13	0.15	0.34	0.11	0.09
	11	0.07	0.08	0.07	0.11	0.23	0.46	0.07	0.26
	12	0.04	0.02	-0.02	0.09	0.10	0.27	0.05	0.21
	13	0.07	0.01	0.07	0.20	0.12	0.53	0.08	0.33
	14	0.04	0.01	-0.02	0.16	0.13	0.40	0.04	0.24
	all	0.11	0.07	0.05	0.07	0.17	0.13	0.11	0.09

Table 23: Precision check on the ConvNet: whole-model cosine with the true gradient at 1024 passes in fp32 with TF32 off, fp32 with TF32 on (PyTorch’s default for convolutions) and fp64, same probes.

Model	Estimator	σ	fp32	TF32	fp64
CIFAR-10 / ConvNet	NP	0.01	0.442	0.440	0.459
	NP	0.001	0.445	0.420	0.475
	ANP	0.01	0.189	0.189	0.193
	ANP	0.001	0.481	0.479	0.482
	INP	0.01	0.442	0.439	0.477
	INP	0.001	0.425	0.358	0.484
	SPSA	0.01	0.115	0.115	0.108
	SPSA	0.001	0.116	0.116	0.110

Table 24: Hyper-parameters shared by all runs.

Optimizer	Adam, $(\beta_1, \beta_2) = (0.9, 0.99)$, $\epsilon_{\text{Adam}} = 10^{-8}$, no schedule, no weight decay (AdamW 10^{-2} in the BP ablation only)
Learning rate $\eta =$ probe scale $\sigma = \varepsilon$	$\{0.1, 0.01, 0.001, 0.0001\}$, 10^{-5} on the image tasks (Table 13) (untied grid in Section 5.3; own recipes in Section 5.4)
Probe distribution	$\mathcal{N}(\mathbf{0}, \mathbf{I})$, scaled by σ
Forward passes per step	16 (probe methods), 17 (1.5-SPSA, 1.5-ANP), 3 / 2 (FF), $1 + 1$ backward (BP)
SPSA, 1.5-SPSA pairs / RS candidates	8 / 15
Curvature exponent α / floor λ	1.5-SPSA: 0.1 / 1.0; 1.5-ANP: 0.5 / 10^{-3}
Batch size	128 images; 64 sequences $\times$ 64 characters
Initialisation	Xavier-uniform, zero biases (LSTM forget gate $= 1$)
FF threshold / PTB corruption	2 (0.25 for the LSTM block) / 30 % of characters
Decorrelation rate η_R (DANP)	10^{-3}, fixed
Training budget	60 min of training time per run, evaluation excluded (30 min in the decoupling ablation)
Evaluation interval	every 1 min (stage 2) / 5 min (stage 1) of training time, and at the end
Divergence rule	non-finite loss, or validation loss $>$ $10\times$ its step-0 value at two consecutive evaluations (stage 1: $2\times$, plus the training-loss clause)
Precision	fp32, TF32 off (stage 2); PyTorch default, TF32 on for convolutions (stage 1)
Seeds	3 (stage 2 and the replicated untied settings), 1 (stage 1, screenings, own-recipe grids and controls)

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and Section 1 claim an empirical comparison of ten training rules on four small architectures under one two-stage protocol with three seeds; this is what Sections 3–5 deliver, and the scope (one hour, ≈40k parameters, single-seed screening) is stated in Section 6.

   Guidelines:

   - The answer NA means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A No or NA answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 6 discusses what the fixed-step final-loss protocol measures, the single-seed screening, the budget and scale, the host dependence of wall-clock results, the deviations of each implementation from its source, and the character-level PTB choice; Section 5.4 states which ingredient of 1.5-ANP the controls do not separate, and Appendix B lists the ablations that were planned but not run.

   Guidelines:

   - The answer NA means that the paper has no limitation while the answer No means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [NA]

   Justification: The paper is purely empirical and contains no theoretical results; the estimator equations in Section 3.2 restate published rules with citations.

   Guidelines:

   - The answer NA means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Sections 3 and 4 and Appendix G specify the architectures, data splits, estimators, budget, optimizer, the two-stage sweep, the selection rule, the divergence rule, the precision and the evaluation protocol; the code, the job lists and all run logs are public.

   Guidelines:

   - The answer NA means that the paper does not include experiments.
   - If the paper includes experiments, a No answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
     (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: The complete code, the job lists of both stages, the per-run JSON results, the checkpoints and the scripts that generate every table and figure are released at `https://github.com/Fchaubard/zero-order-ablations`; the datasets are public and downloaded by the code.

   Guidelines:

   - The answer NA means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://nips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so "No" is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://nips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

   Answer: [Yes]

   Justification: Section 4 gives the data splits, batch sizes, initialisation, optimizer, the ε screening and the validation-based selection rule; Appendix G (Table 24) lists every hyperparameter.

   Guidelines:

   - The answer NA means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

Justification: The main tables report the mean over three seeds with the sample standard deviation ($n = 3$, one sigma) in units of the last digit; cells without a parenthesis are single runs (the few pairs whose selected ε lies at the grid edge until their seeds complete), which the captions mark, and the mean ranks carry a bootstrap interval over seeds. On CIFAR-10 the seed standard deviations (0.05 in loss, two points in accuracy) do not resolve the order within the node-perturbation family, which the text states.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The authors should answer "Yes" if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g. negative error rates).
- If error bars are reported in tables or plots, The authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Section 4 states that every run uses one RTX 5090 for one hour of training (plus evaluation) on an 8-GPU node; Appendix G gives the total compute of both stages, the ablations and the diagnostics, including the preliminary runs.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: [Yes]

Justification: The study trains small models on three standard public benchmarks; it involves no human subjects, no personal data and no deployment.

Guidelines:

- The answer NA means that the authors have not reviewed the NeurIPS Code of Ethics.

- If the authors answer No, they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [NA]

    Justification: This is foundational optimizer-benchmarking research on ≈40k-parameter models with no direct application or deployment path; its potential positive impact (more energy-efficient or hardware-friendly training) is discussed in Section 1 and we see no specific negative impact beyond those of generic training algorithms.

    Guidelines:

    - The answer NA means that there is no societal impact of the work performed.
    - If the authors answer NA or No, they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pretrained language models, image generators, or scraped datasets)?

    Answer: [NA]

    Justification: No models with misuse potential are released; the released artefacts are training code, logs and ≈40k-parameter checkpoints on MNIST, CIFAR-10 and character-level PTB.

    Guidelines:

    - The answer NA means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: MNIST, CIFAR-10 and Penn Treebank are cited (Section 4) and their terms of use are stated in Appendix G; the reference implementations of the node-perturbation family, forward-forward and 1.5-SPSA are cited where each method is defined; PyTorch is used under its BSD licence.

    Guidelines:

    - The answer NA means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: The released code repository (MIT licence) documents the two-stage protocol (`PROTOCOL.md`), the job lists, the command to launch every run and the scripts that regenerate every table and figure; the result files of both stages, the ablations and the diagnostics are included.

    Guidelines:

    - The answer NA means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [NA]

    Justification: The paper involves neither crowdsourcing nor human subjects.

    Guidelines:

    - The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.

- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [NA]

    Justification: The paper involves no human subjects, so no IRB review was required.

    Guidelines:

    - The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does not impact the core methodology, scientific rigorousness, or originality of the research, declaration is not required.

    Answer: [NA]

    Justification: LLM-based assistants were used only for writing and coding assistance; no LLM is part of the methods, the experiments or the analysis.

    Guidelines:

    - The answer NA means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy (`https://neurips.cc/Conferences/2025/LLM`) for what should or should not be described.